# SOFTWARE ARCHITECTURE DOCUMENT (SAD)

S.L.O.T.H: An optimized ETL-based architecture for migrating business logic contained in spreadsheets

**Autores**

Diederik Antonio Montaño Burbano
Juan Diego Pérez Navarro
Mauro Alonzgo Gonzáles Figueroa

**Tutores**

Isaac Zúñiga Silgado
Jairo Enrique Serrano Castañeda

**UNIVERSIDAD TECNOLÓGICA DE BOLÍVAR**

Escuela de Transformación Digital
Proyecto de Ingeniería II

Versión: 1.0

Fecha: 13/05/2026
Cartagena de Indias, D.T y C, Colombia

# Índice

1. Introducción ..... 6
2. Contexto y alcance ..... 8
2.1 Contexto funcional ..... 8
2.2 Alcance del proyecto ..... 8
2.3 Apoyos de lectura ..... 10
2.4 Glosario ..... 11
3. Requerimientos ..... 16
3.1 Requerimientos funcionales ..... 16
3.1.1 Registro e Inicio de Sesión de Usuarios ..... 16
3.1.2 Administración de usuarios y perfiles ..... 16
3.1.3 Control de Permisos por Rol y Recurso ..... 16
3.1.4 Gestión de Proyectos y Credenciales de BD Destino ..... 17
3.1.5 Gestión de Miembros y Colaboración por Proyecto ..... 17
3.1.6 Creación y Actualización de Esquemas de Validación ..... 17
3.1.7 Consulta y Reversión de Versiones de Esquemas ..... 17
3.1.8 Validación Asíncrona de Archivos ..... 18
3.1.9 Inserción Asíncrona en Base de Datos Destino ..... 18
3.1.10 Flujo Combinado de Procesamiento Integral ..... 18
3.1.11 Generación de SQL desde Excel con Fórmulas ..... 19
3.1.12 Creación Dinámica de Tablas desde Esquemas o Archivos ..... 19
3.1.13 Consulta de Estado y Resultados de Tareas ..... 19
3.1.14 Monitoreo de Salud y Métricas del Sistema ..... 19
3.2 Requerimientos no funcionales ..... 20
3.2.1 Seguridad y privacidad ..... 20
3.2.2 Rendimiento y escalabilidad ..... 21
3.2.3 Disponibilidad y resiliencia ..... 22
3.2.4 Mantenibilidad y extensibilidad ..... 22
4. Arquitectura ..... 23
4.1 Propósito de la arquitectura ..... 23
4.2 Estilo arquitectónico ..... 24
4.3 Principios de diseño ..... 24

4.3.1 Responsabilidad única por servicio ..... 24
4.3.2 Contratos explícitos entre servicios ..... 25
4.3.3 Idempotencia operativa ..... 25
4.3.4 Externalización del estado ..... 25
4.3.5 Caché como acelerador, no como fuente de datos determinante ..... 25
4.3.6 Trazabilidad extremo a extremo ..... 25
4.4 Vista general de componentes ..... 25
4.5 Vista lógica de la API central ..... 27
4.6 Microservicios y responsabilidades ..... 27
4.6.1 Core API service ..... 27
4.6.2 Database Service ..... 28
4.6.3 Typechecking Service ..... 28
4.6.4 Parsing Subsystem ..... 28
4.6.5 Web App ..... 29
4.7 Interacciones y protocolos ..... 29
4.8 Vista de flujos críticos de negocio ..... 30
4.8.1 Gestión de reglas de validación ..... 30
4.8.2 Validación asincrónica de carga ..... 31
4.8.3 Inserción asincrónica ..... 32
4.8.4 Migración de fórmulas a SQL ..... 32
4.9 Estados y consistencia ..... 33
4.10 Modelos de datos arquitectónicamente relevantes ..... 33
4.10.1 Entidades relacionales ..... 33
4.10.2 Estructuras documentales ..... 34
5. Infraestructura y despliegue ..... 34
5.1 Propósito de la infraestructura ..... 34
5.2 Enfoque general ..... 35
5.3 Infraestructura física ..... 35
5.3.1 Aprovisionamiento en AWS ..... 35
5.3.2 Roles de nodos del clúster ..... 37
5.3.3 Almacenamiento compartido ..... 38
5.4 Aprovisionamiento e inicialización del clúster ..... 38
5.4.1 Aprovisionamiento con terraform ..... 38

5.4.2 Configuración del sistema utilizando Ansible ..... 38
5.5 Orquestación de servicios con Docker Swarm ..... 39
5.5.1 Justificación ..... 39
5.5.2 Definición de stacks por ambiente ..... 39
5.5.3 Servicios desplegados ..... 40
5.5.4 Políticas operativas de Swarm ..... 41
5.6 Segmentación de red ..... 41
5.7 Seguridad operativa ..... 42
5.7.1 Seguridad de red ..... 42
5.7.2 Seguridad de acceso y certificados ..... 42
5.7.3 Seguridad del plano de gestión ..... 42
5.7.4 Gestión de secretos y variables de entorno ..... 42
5.8 Persistencia y datos operacionales ..... 43
5.8.1 PostgreSQL ..... 43
5.8.2 MongoDB ..... 43
5.8.3 Redis ..... 43
5.8.4 RabbitMQ ..... 43
5.9 Observabilidad y operación ..... 43
5.9.1 Recolección de métricas ..... 44
5.9.2 Logs y trazas ..... 44
5.9.3 Visualización operacional ..... 44
5.9.4 Autoscaling ..... 44
5.10 Pipeline de entrega continua ..... 45
5.10.1 Integración continua ..... 45
5.10.2 Versionado y promoción entre ambientes ..... 45
5.10.3 Despliegue remoto al clúster ..... 45
5.11 Limitaciones y consideraciones actuales ..... 47
6. Decisiones y criterios relevantes ..... 48
6.1 Arquitectura ..... 48
6.1.1 Adoptar infraestructura basada en microservicios ..... 48
6.1.2 Separar operaciones sincrónicas y asincrónicas ..... 49
6.1.3 Utilizar RabbitMQ como centro para el procesamiento asincrónico ..... 49
6.1.4 Utilizar gRPC para comunicación interna entre servicios core ..... 50

6.1.5 Almacenar reglas de validación en Json schemas ..... 50
6.1.6 Almacenamiento y versionamiento de reglas en MongoDB ..... 51
6.1.7 Uso de Redis como caché operacional ..... 51
6.1.8 Diseño de workers sin estado persistente propio ..... 51
6.1.9 Incorporar idempotencia como principio base del pipeline ..... 52
6.1.10 Priorizar disponibilidad y tolerancia a particiones en procesos asincrónicos ..... 52
6.2 Infraestructura ..... 53
6.2.1 Uso de infraestructura como código ..... 53
6.2.2 Aprovisionamiento sobre AWS ..... 53
6.2.3 Separar en mútiples nodos manager y workers en Docker Swarm ..... 54
6.2.4 Usar Docker Swarm como orquestador ..... 54
6.2.5 Usar Traefik como punto de entrada y reverse proxy ..... 54
6.2.6 Segmentar redes overlay por dominio funcional ..... 54
6.2.7 Utilizar EFS como almacenamiento compartido ..... 54
6.2.8 Observabilidad extensiva en la infraestructura base ..... 55
6.3 Criterios transversales de evaluación ..... 55
7. Resultados ..... 55
7.1 Resultados funcionales ..... 56
7.2 Resultados de validación de la interfaz no-code ..... 56
7.3 Resultados de rendimiento y escalabilidad ..... 57
7.4 Resultados del sistema de parsing y migración de lógica ..... 59
7.5 Resultados arquitectónicos ..... 60
7.7 Síntesis de resultados ..... 60
8 Conclusiones ..... 60
8.1 Limitaciones ..... 61
8.2 Trabajo futuro ..... 62

# 1. Introducción

El Sistema para la Orquestación, Traducción y Manejo de Lógica, o por su acrónimo en inglés, S.L.O.T.H, es un producto derivado de la línea de investigación de computación de alto rendimiento de la Universidad Tecnológica de Bolívar (UTB). Surge como respuesta a una problemática recurrente en los ecosistemas de Business Intelligence y Data Warehousing: la dificultad de migrar, validar, auditar y gobernar lógica de negocio contenida en fuentes de datos manejadas por usuarios no técnicos, especialmente hojas de cálculo en formatos como .xlsx y .csv.

Las grandes organizaciones generan, consumen y almacenan vastos volúmenes de datos provenientes de múltiples fuentes heterogéneas. Estos datos deben ser procesados, transformados y consolidados en repositorios analíticos que permitan la generación de reportes, la auditoría de información y la toma de decisiones estratégicas. En este contexto, los procesos ETL (extracción, transformación y carga) constituyen un componente fundamental, pues actúan como puente entre los datos operacionales sin procesar y el conocimiento útil almacenado en un data warehouse.

Sin embargo, en entornos empresariales reales, una parte significativa de la información que alimenta estos procesos proviene de hojas de cálculo creadas, modificadas y cargadas manualmente por usuarios de distintas áreas de negocio. Aunque estas herramientas son ampliamente adoptadas por su flexibilidad, accesibilidad y facilidad de uso, también introducen riesgos técnicos importantes: inconsistencias estructurales, errores humanos, formatos variables, celdas combinadas, datos faltantes y reglas de negocio implícitas o dispersas. En muchos casos, dichas reglas se encuentran embebidas directamente en scripts ETL o encapsuladas en fórmulas de hojas de cálculo, lo que dificulta su trazabilidad, mantenimiento, versionamiento y reutilización.

S.L.O.T.H. propone una arquitectura ETL resiliente, distribuida, escalable y de alta disponibilidad, orientada a la centralización de reglas de validación y a la migración de lógica de negocio contenida en hojas de cálculo hacia una infraestructura controlada. Para ello, el sistema desacopla las reglas de negocio del código fuente, las almacena en un repositorio centralizado basado en MongoDB y las representa mediante JSON Schema, permitiendo su gestión, auditoría y versionamiento de forma independiente al ciclo de desarrollo del software. Esta aproximación reduce la dependencia de equipos técnicos para realizar ajustes menores en reglas de validación y habilita un modelo de operación más ágil, gobernable y transparente.

Adicionalmente, el sistema incorpora una interfaz gráfica no-code para la definición y administración de reglas por parte de usuarios autorizados, un motor dinámico de validación y carga, un sistema de procesamiento paralelo basado en colas de tareas y un servicio de parsing capaz de interpretar fórmulas presentes en archivos .xlsx. Este último componente utiliza estructuras como Abstract Syntax Trees y grafos de dependencia para traducir fórmulas de hojas de cálculo a instrucciones SQL, permitiendo que la lógica computacional sea migrada y ejecutada directamente en el motor de base de datos del data warehouse.

Para asegurar disponibilidad, escalabilidad y una enorme resiliencia, S.L.O.T.H se encuentra construido en una arquitectura de microservicios, soportados por una infraestructura basada en el despliegue de contenedores Docker, con comunicación entre servicios mediante gRPC y procesamiento asincrónico utilizando RabbitMQ. Esta organización permite escalar vertical y horizontalmente los componentes responsables de la validación, carga, traducción y creación de estructuras en la base de datos. Las

evaluaciones realizadas en los trabajos previos muestran mejoras relevantes en los tiempos de procesamiento, incluyendo reducciones de hasta el 70 % en la fase de validación mediante paralelización y mejoras adicionales al migrar parte de la lógica de cálculo desde las hojas de cálculo hacia el data warehouse.

En consecuencia, este documento de arquitectura de software describe la estructura, componentes, decisiones técnicas, responsabilidades, restricciones y mecanismos de interacción que conforman S.L.O.T.H. Su propósito es servir como referencia técnica para comprender la solución, facilitar su evolución, orientar futuras implementaciones y documentar las decisiones arquitectónicas que permiten al sistema cumplir con sus objetivos de escalabilidad, gobernanza de datos, trazabilidad, mantenibilidad y eficiencia en la migración de lógica de negocio.

# 2. Contexto y alcance

Esta sección define el contexto funcional en el que se ubica S.L.O.T.H., el alcance del proyecto en su versión actual y los principales términos utilizados a lo largo del documento. Su propósito es delimitar con claridad qué problema aborda el sistema, qué capacidades cubre la solución propuesta y qué elementos quedan fuera del alcance inmediato del proyecto.

## 2.1 Contexto funcional

Funcionalmente, S.L.O.T.H. se ubica entre los usuarios que producen o cargan archivos tabulares y los sistemas analíticos que consumen datos consolidados. Su papel es actuar como una capa intermedia de validación, gobierno, transformación y carga, evitando que la información proveniente de hojas de cálculo sea incorporada directamente al repositorio analítico sin controles previos.

El flujo general inicia cuando un usuario autorizado accede a la plataforma, crea o selecciona un proyecto y configura las credenciales de conexión hacia una base de datos destino. Dentro de ese proyecto, el usuario puede definir esquemas de validación asociados a tablas específicas, utilizando reglas representadas mediante JSON Schema. Estas reglas establecen las condiciones que deben cumplir los datos antes de ser insertados, tales como tipos de datos, obligatoriedad de campos, rangos permitidos, formatos esperados o restricciones estructurales.

Una vez definidos los esquemas, el usuario puede cargar archivos tabulares en formatos como .xlsx, .xls o .csv. El sistema registra la solicitud como una tarea, calcula mecanismos de idempotencia para evitar reprocesamientos duplicados y delega el procesamiento a una cola de mensajería. A partir de este punto, los workers especializados validan el archivo contra el esquema activo, identifican registros válidos e inválidos, generan resultados de validación y, cuando corresponde, ejecutan la inserción de datos en la base destino.

En los casos donde el archivo contiene fórmulas de Excel, S.L.O.T.H. incorpora un subsistema de parsing encargado de extraer dichas fórmulas, construir representaciones intermedias como árboles de sintaxis abstracta y traducirlas a sentencias SQL compatibles con PostgreSQL. De esta manera, el sistema no se limita a mover datos entre archivos y bases de datos, sino que también busca migrar lógica computacional desde las hojas de cálculo hacia un entorno centralizado, auditable y reproducible.

El sistema también contempla capacidades de consulta de estado, monitoreo de tareas, revisión de resultados, versionamiento de esquemas y trazabilidad de operaciones. Estas funciones permiten que el usuario conozca el estado de una carga, identifique errores, revise resultados históricos y mantenga control sobre la evolución de las reglas de validación aplicadas a cada proyecto.

## 2.2 Alcance del proyecto

El alcance de S.L.O.T.H. comprende el diseño e implementación de una plataforma distribuida para la gestión, validación, traducción y carga de información proveniente de hojas de cálculo hacia bases de datos destino. La solución cubre tanto componentes de interacción con el usuario como servicios internos de procesamiento, persistencia, comunicación y orquestación asincrónica.

Dentro del alcance del proyecto se incluyen las siguientes capacidades:

**Gestión de usuarios y autenticación:** El sistema permite registrar usuarios, iniciar sesión y controlar el acceso a las funcionalidades de la plataforma. La autenticación se apoya en credenciales de usuario y mecanismos de sesión que permiten identificar al actor que ejecuta cada operación.

**Administración de proyectos:** El sistema permite crear y administrar proyectos como unidades de trabajo. Cada proyecto agrupa usuarios, configuraciones, credenciales de conexión, esquemas de validación, archivos procesados y tareas asociadas. Esta organización permite separar contextos de procesamiento y aplicar reglas de autorización por proyecto.

**Gestión de miembros y permisos:** El sistema permite asociar usuarios a proyectos mediante roles o niveles de acceso. Esta capacidad habilita la colaboración entre múltiples usuarios sobre un mismo conjunto de recursos, manteniendo restricciones sobre quién puede consultar, modificar, cargar archivos o administrar configuraciones críticas.

**Configuración de bases de datos destino:** El sistema permite almacenar parámetros de conexión hacia bases de datos destino, incluyendo host, usuario, contraseña y demás datos necesarios para ejecutar operaciones de creación de tablas e inserción. Las credenciales sensibles deben manejarse bajo mecanismos de protección definidos en los requerimientos no funcionales.

**Creación y actualización de esquemas de validación:** El sistema permite definir esquemas de validación mediante JSON Schema. Estos esquemas representan las reglas que deben cumplir los datos antes de ser aceptados para carga. El sistema también permite actualizar esquemas existentes y mantener trazabilidad sobre los cambios realizados.

**Versionamiento y reversión de esquemas:** El sistema mantiene un historial de versiones de los esquemas de validación, permitiendo consultar versiones anteriores y revertir un esquema cuando sea necesario. Esta capacidad es fundamental para la gobernanza de reglas de negocio y para la recuperación ante cambios incorrectos.

**Validación asincrónica de archivos:** El sistema permite validar archivos tabulares de forma asincrónica mediante una cola de mensajes y workers especializados. Esta capacidad evita que la interfaz de usuario quede bloqueada durante procesos de larga duración y permite distribuir el procesamiento en función de la carga del sistema.

**Inserción asincrónica en base de datos destino:** El sistema permite insertar datos válidos en la base de datos configurada para el proyecto. La inserción se ejecuta como parte de un flujo asincrónico, posterior a la validación, y debe registrar el estado final de la operación para consulta posterior.

**Procesamiento integral de archivos:** El sistema permite ejecutar un flujo combinado que encadena validación, inserción y generación de resultados en una sola operación lógica para el usuario. Aunque internamente el proceso se divide en etapas, externamente se presenta como una tarea rastreable mediante un identificador único.

**Generación de SQL a partir de archivos Excel con fórmulas:** El sistema permite analizar archivos Excel que contienen fórmulas y generar sentencias SQL equivalentes cuando dichas fórmulas son compatibles con las capacidades implementadas por el subsistema de parsing. Esta funcionalidad busca trasladar lógica de cálculo desde el archivo hacia la base de datos destino.

**Creación dinámica de tablas:** El sistema permite crear tablas en la base de datos destino a partir de esquemas definidos o de estructuras detectadas en los archivos cargados. Esta capacidad facilita la incorporación de nuevos formatos de datos sin requerir intervención manual directa sobre la base de datos.

**Consulta de estado y resultados de tareas:** El sistema permite consultar el estado de tareas asincrónicas mediante identificadores únicos. Los estados pueden incluir valores como pendiente, en procesamiento, completado o fallido, acompañados de resultados, errores o metadatos relevantes.

**Monitoreo técnico del sistema:** El sistema expone mecanismos de salud y métricas para verificar el estado operativo de los servicios, la conectividad con dependencias y el comportamiento general de la plataforma.

Aunque estas capacidades conforman el alcance principal del proyecto, también es necesario delimitar explícitamente aquello que no forma parte del alcance actual. S.L.O.T.H. no pretende reemplazar completamente una plataforma empresarial de Business Intelligence ni proveer herramientas avanzadas de visualización analítica. Tampoco busca cubrir la totalidad de funciones existentes en Excel, sino soportar un conjunto progresivo de fórmulas traducibles a SQL. Asimismo, la solución no garantiza compatibilidad universal con cualquier motor de base de datos, dado que el flujo principal está orientado a PostgreSQL como destino inicial.

El sistema tampoco tiene como objetivo corregir automáticamente todos los errores presentes en los archivos cargados. Su responsabilidad es detectarlos, reportarlos y evitar que datos inválidos sean insertados, dejando la corrección final en manos del usuario o del proceso de negocio correspondiente.

## 2.3 Apoyos de lectura

Al ser un SAD, este documento cuenta con un lenguaje profundamente técnico, referencias a código y diagramas extensos. Para facilitar la lectura, hemos añadido algunos apoyos de lectura cuya revisión recomendamos antes y durante la lectura:

1. **Glosario**: Un resumen de tecnicismos y acrónimos utilizados de manera constante en el documento junto a sus definiciones.

2. **Carpeta de imágenes externa:** Algunos diagramas son extensos y no pueden ser visualizados de manera adecuada en este documento, por lo que puede hacer click en sus descripciones (al pie de cada imagen) para ser llevado a una carpeta de OneDrive pública con las imágenes en formato SVG o PDF de alta definición.

3. **Artículos relacionados:** Investigaciones previas a este documento, cuya lectura previa recomendamos encarecidamente. Puede dirigirse a ellos haciendo *click aquí*.

4. **Conexiones al código:** Cada referencia a un archivo del código fuente puede ser presionada y se le redirigirá directamente a su ubicación en el repositorio de S.L.O.T.H en la organización ISCOUTB.

## 2.4 Glosario

En esta sección encontrará un conjunto de términos utilizados de manera constante en el resto del documento. Es recomendable darle una lectura inicial y volver a él en cuanto se encuentre con palabras o conceptos desconocidos.

**S.L.O.T.H.**: Acrónimo de *System for Logic Orchestration, Translation and Handling*. Corresponde al sistema desarrollado para orquestar, traducir y gestionar lógica de negocio contenida en hojas de cálculo dentro de un flujo ETL distribuido.

**ETL:** Proceso de extracción, transformación y carga de datos. En S.L.O.T.H., este proceso se aplica a archivos tabulares que deben ser validados, transformados y cargados hacia una base de datos destino.

**Data warehouse:** Repositorio analítico utilizado para almacenar datos consolidados provenientes de distintas fuentes. En el contexto del sistema, representa el destino final o conceptual de los datos procesados.

**Hoja de cálculo:** Archivo tabular, usualmente en formato .xlsx, .xls o .csv, que contiene datos organizados en filas y columnas. En algunos casos puede incluir fórmulas que representan lógica de negocio.

**Archivo tabular:** Archivo estructurado en forma de tabla. Incluye archivos Excel y CSV procesados por la plataforma.

**JSON Schema**: Formato utilizado para describir la estructura esperada de un documento JSON y definir reglas de validación sobre sus campos. En S.L.O.T.H. se utiliza para representar las restricciones que deben cumplir los datos cargados.

**Esquema de validación:** Conjunto de reglas asociadas a una tabla o entidad destino. Define los tipos de datos, campos obligatorios, restricciones y condiciones necesarias para aceptar registros durante el proceso de carga.

**Regla de negocio:** Condición, restricción o cálculo que representa una decisión funcional del dominio. Puede estar expresada como regla de validación, fórmula de Excel o transformación aplicada a los datos.

**Proyecto:** Unidad lógica de trabajo dentro de S.L.O.T.H. Agrupa usuarios, credenciales de base de datos, esquemas, archivos, tareas y configuraciones asociadas a un mismo contexto de procesamiento.

**Base de datos destino**
Base de datos configurada dentro de un proyecto para recibir las tablas y datos procesados por el sistema.

**Validación:** Proceso mediante el cual el sistema verifica que los registros de un archivo cumplen las reglas definidas en el esquema activo correspondiente.

**Inserción:** Proceso mediante el cual los registros válidos son cargados en la base de datos destino.

**Parsing:** Proceso de análisis estructural de fórmulas o expresiones. En S.L.O.T.H. se utiliza para interpretar fórmulas de Excel y convertirlas en representaciones intermedias que puedan traducirse a SQL.

**AST:** Sigla de *Abstract Syntax Tree* o árbol de sintaxis abstracta. Es una estructura jerárquica que representa la organización lógica de una fórmula o expresión.

**SQL:** Lenguaje utilizado para definir, consultar y manipular datos en bases de datos relacionales. En S.L.O.T.H. se utiliza para crear tablas, insertar registros y representar fórmulas migradas desde Excel.

**DDL:** Subconjunto de SQL orientado a la definición de estructuras de base de datos, como tablas, columnas y restricciones.

**DML:** Subconjunto de SQL orientado a la manipulación de datos, como inserciones, actualizaciones o eliminaciones.

**RabbitMQ:** Sistema de mensajería utilizado para coordinar tareas asincrónicas. En S.L.O.T.H. permite desacoplar la recepción de archivos del procesamiento ejecutado por workers.

**Worker:** Componente encargado de consumir tareas desde una cola y ejecutar una operación específica, como validación, inserción o consolidación de resultados.

**Core API:** Servicio principal de negocio encargado de exponer endpoints, autenticar usuarios, validar permisos, registrar tareas, publicar mensajes y coordinar operaciones entre componentes.

**Web App:** Interfaz de usuario del sistema. Permite gestionar proyectos, esquemas, cargas, resultados y consultas de estado.

**Database Service:** Servicio interno que unifica el acceso a Redis y MongoDB mediante contratos gRPC. Se utiliza para consultar y persistir estados, esquemas, versiones y datos operacionales.

**Redis:** Sistema de almacenamiento en memoria utilizado como caché y mecanismo de acceso rápido a información operacional.

**MongoDB:** Base de datos documental utilizada para almacenar esquemas activos, versiones de esquemas y estructuras flexibles relacionadas con reglas de validación.

**PostgreSQL:** Base de datos relacional utilizada para persistir entidades transaccionales del sistema y como motor destino inicial para las operaciones de creación de tablas e inserción.

**gRPC:** Protocolo de comunicación utilizado entre servicios internos que requieren contratos tipados, baja latencia y eficiencia en la serialización.

**AMQP:** Protocolo de mensajería utilizado por RabbitMQ para coordinar tareas asincrónicas entre servicios.

**Idempotencia:** Propiedad que permite ejecutar una operación más de una vez sin producir efectos duplicados. En S.L.O.T.H. se utiliza para evitar que la carga repetida de un mismo archivo genere duplicidad de datos.

**Task ID:** Identificador único asociado a una tarea asincrónica. Permite consultar el estado y resultado de operaciones como validación, inserción o procesamiento integral.

**RBAC:** Modelo de control de acceso basado en roles. Permite definir qué acciones puede ejecutar un usuario según su rol dentro del sistema o proyecto.

**ABAC:** Modelo de control de acceso basado en atributos. Permite tomar decisiones de autorización considerando propiedades del usuario, recurso, acción o contexto.

**Teorema CAP:** Principio de sistemas distribuidos que establece que, ante una partición de red, un sistema distribuido debe priorizar entre consistencia y disponibilidad.

**Consistencia eventual:** Modelo de consistencia en el que los distintos componentes del sistema pueden observar estados temporales diferentes durante una ventana corta, pero convergen posteriormente hacia un estado final coherente.

**Fuente de verdad:** Repositorio considerado autoritativo para un tipo específico de información.

**Estado transaccional:** Información que requiere consistencia fuerte, relaciones claras y persistencia formal, como usuarios, proyectos, membresías y tareas de carga.

**Estado operacional:** Información asociada al progreso, coordinación o ejecución temporal de procesos distribuidos, como estados parciales de tareas, datos en caché o resultados intermedios.

**Pipeline:** Secuencia de etapas encadenadas que procesan una entrada hasta producir un resultado.

**Cola de mensajes:** Estructura utilizada para almacenar tareas pendientes de procesamiento asincrónico. Permite desacoplar el servicio que publica una tarea del worker que la consume.

**Ack/Nack:** Mecanismos de confirmación o rechazo de mensajes en una cola. Un ack indica que el mensaje fue procesado correctamente; un nack indica que debe rechazarse, reintentarse o enviarse a una cola de error.

**Reintento:** Ejecución repetida de una tarea que falló por una causa transitoria, como indisponibilidad temporal de red, base de datos o servicio.

**Dead Letter Queue:** Cola destinada a almacenar mensajes que no pudieron procesarse correctamente después de agotar los reintentos definidos.

**Callback:** Llamada realizada por un servicio para notificar a otro que una operación finalizó.

**Webhook:** Endpoint expuesto para recibir notificaciones automáticas desde otro sistema o servicio. Es conceptualmente cercano al callback, pero suele usarse como mecanismo de integración externa.

**JWT:** Token utilizado para representar la identidad autenticada de un usuario y transportar información necesaria para autorizar solicitudes.

**TLS:** Protocolo de seguridad utilizado para cifrar comunicaciones en tránsito, especialmente en conexiones HTTP/HTTPS expuestas mediante Traefik.

**Reverse proxy:** Componente que recibe tráfico externo y lo enruta hacia servicios internos.

**Traefik:** Reverse proxy utilizado como punto de entrada HTTP/HTTPS, terminador TLS y enrutador dinámico de servicios dentro del clúster.

**Docker Swarm:** Orquestador de contenedores utilizado para desplegar, replicar, actualizar y administrar servicios distribuidos en el clúster. El documento lo utiliza como plataforma activa de despliegue.

**Manager node:** Nodo de Docker Swarm encargado de mantener el estado del clúster y coordinar operaciones administrativas.

**Worker node:** Nodo de Docker Swarm encargado de ejecutar cargas de aplicación y procesamiento, como APIs, servicios de parsing o workers ETL.

**Red overlay:** Red virtual distribuida que permite la comunicación entre contenedores ejecutados en distintos nodos del clúster.

**Terraform:** Herramienta de infraestructura como código utilizada para aprovisionar recursos base en AWS, como VPC, subredes, EC2, Security Groups, IAM y EFS.

**Ansible:** Herramienta de automatización utilizada para configurar instancias, instalar Docker, montar almacenamiento compartido e inicializar el clúster.

**GitHub Actions:** Plataforma de automatización utilizada para ejecutar integración continua, construir imágenes, publicar artefactos y desplegar servicios.

**CI/CD:** Práctica de integración y entrega continua que permite validar, construir, versionar y desplegar cambios de forma automatizada.

**IaC:** Sigla de *Infrastructure as Code*. Enfoque donde la infraestructura se define mediante archivos versionados y reproducibles, en lugar de configuraciones manuales.

**EFS:** Servicio de almacenamiento compartido de AWS utilizado para que varios nodos accedan a archivos comunes, como certificados o artefactos operativos.

**VPC:** Red virtual privada en AWS utilizada para aislar recursos de infraestructura dentro de un espacio de red controlado.

**Security Group:** Mecanismo de control de tráfico en AWS que define qué puertos y orígenes pueden acceder a los recursos.

**IAM:** Servicio de AWS utilizado para administrar identidades, permisos y perfiles de acceso a recursos de infraestructura.

**Observabilidad:** Capacidad de entender el comportamiento interno del sistema mediante métricas, logs y trazas.

**Métrica:** Valor cuantitativo utilizado para evaluar el comportamiento del sistema, como uso de CPU, memoria, duración de tareas, errores o tamaño de colas.

**Log:** Registro textual de eventos, errores o acciones producidas por un servicio durante su ejecución.

**Traza distribuida:** Registro del recorrido de una solicitud o tarea a través de múltiples servicios, útil para diagnosticar fallos en sistemas distribuidos.

**Prometheus:** Herramienta utilizada para recolectar métricas de servicios, contenedores y nodos.

**Grafana:** Herramienta utilizada para visualizar métricas, logs y trazas mediante dashboards.

**Loki:** Sistema utilizado para centralizar y consultar logs.

**Tempo:** Sistema utilizado para almacenar y consultar trazas distribuidas.

**Autoscaling:** Capacidad de ajustar automáticamente el número de réplicas de un servicio según métricas de carga o rendimiento.

# 3. Requerimientos

## 3.1 Requerimientos funcionales

A continuación, se describen los servicios y procesos que el sistema ejecuta, detallando los flujos de información necesarios para cada operación.

### 3.1.1 Registro e Inicio de Sesión de Usuarios

El sistema debe garantizar un punto de entrada seguro para los usuarios, implementando un flujo de registro y autenticación que valida la integridad de las credenciales antes de otorgar acceso.

| | |
|---|---|
| **Entradas** | `username`, `email`, `password` (vía `POST /api/v1/auth/sign-up` o `sign-in`). |
| **Acción** | El sistema valida el formato de los datos, verifica la existencia previa del usuario y autentica las credenciales contra el almacenamiento relacional. |
| **Salidas** | Objeto de usuario autenticado con token de sesión o mensajes de error detallados. |

### 3.1.2 Administración de usuarios y perfiles

Permite la gestión continua de los usuarios dentro de la plataforma, asegurando que la información esté actualizada y los roles correctamente asignados.

| | |
|---|---|
| **Entradas** | Parámetros de consulta o datos de perfil (vía endpoints `/api/v1/users/*`). |
| **Acción** | Ejecución de operaciones CRUD sobre la entidad usuario, aplicando reglas de negocio y restricciones de seguridad. |
| **Salidas** | Datos del usuario en formato JSON o respuestas paginadas para búsquedas. |

### 3.1.3 Control de Permisos por Rol y Recurso

El sistema evalúa dinámicamente cada solicitud para asegurar que el usuario tenga los privilegios necesarios.

| | |
|---|---|
| **Entradas** | Identidad del usuario (JWT), acción solicitada y recurso objetivo (ej: un proyecto específico). |
| **Acción** | Evaluación de reglas RBAC/ABAC sobre el contexto de la solicitud. |
| **Salidas** | Autorización explícita (`permitir/denegar`) para el acceso al endpoint. |

### 3.1.4 Gestión de Proyectos y Credenciales de BD Destino

El sistema debe gestionar proyectos que funcionen como contenedores de trabajo donde se definen las conexiones a las bases de datos destino.

| | |
|---|---|
| **Entradas** | Nombre del proyecto y parámetros de conexión (`db_host`, `db_user`, `db_password`, etc.). |
| **Acción** | Creación y persistencia de la configuración del proyecto con cifrado de credenciales sensibles. |
| **Salidas** | ID único del proyecto y confirmación de almacenamiento. |

### 3.1.5 Gestión de Miembros y Colaboración por Proyecto

Permite la interacción entre múltiples usuarios sobre un mismo conjunto de recursos, definiendo niveles de jerarquía.

| | |
|---|---|
| **Entradas** | ID de usuario a invitar e ID del proyecto. |
| **Acción** | Asociación de usuarios a proyectos con roles específicos (`owner`, `shared`, `viewer`). |
| **Salidas** | Lista actualizada de miembros del proyecto con sus respectivos niveles de acceso. |

### 3.1.6 Creación y Actualización de Esquemas de Validación

El sistema permite definir reglas de integridad mediante JSON Schema para las tablas que serán receptoras de datos.

| | |
|---|---|
| **Entradas** | Nombre de la tabla y definición del JSON Schema (vía `POST /api/v1/schemas/{project_id}`). |
| **Acción** | Validación del esquema propuesto y persistencia en MongoDB, manteniendo la trazabilidad del cambio. |
| **Salidas** | Confirmación del estado (`inserted`, `updated` o `no_change`). |

### 3.1.7 Consulta y Reversión de Versiones de Esquemas

Para garantizar la estabilidad, el sistema permite navegar por el historial de cambios de un esquema.

| | |
|---|---|
| **Entradas** | ID del proyecto y nombre de la tabla (opcionalmente un ID de versión). |
| **Acción** | Recuperación del esquema activo o reversión a una versión previa almacenada en el historial. |
| **Salidas** | Documento JSON Schema correspondiente a la versión solicitada. |

### 3.1.8 Validación Asíncrona de Archivos

Permite verificar la calidad de grandes volúmenes de datos sin interrumpir el flujo de trabajo del usuario.

| | |
|---|---|
| **Entradas** | Archivo tabular (`.xlsx`, `.xls`, `.csv`), `project_id` y `table_name`. |
| **Acción** | Registro de una tarea de validación, generación de clave de idempotencia y envío del mensaje a la cola de RabbitMQ para procesamiento paralelo. |
| **Salidas** | `HTTP 202` con el `task_id` para seguimiento. |

### 3.1.9 Inserción Asíncrona en Base de Datos Destino

Coordina la carga de datos previamente validados en la infraestructura de base de datos del cliente.

| | |
|---|---|
| **Entradas** | `task_id` de una validación exitosa o archivo directo con parámetros de inserción. |
| **Acción** | El worker solicita la generación de SQL, abre una conexión segura y ejecuta las sentencias de inserción en la BD destino. |
| **Salidas** | Estado final de la tarea (`success/error`) persistido y disponible para consulta. |

### 3.1.10 Flujo Combinado de Procesamiento Integral

Encadena los procesos de validación e inserción en una sola operación lógica para el usuario.

| | |
|---|---|
| **Entradas** | Archivo de origen y parámetros de destino completos (vía `POST /api/v1/uploads/process`). |
| **Acción** | Orquestación secuencial de workers: Validación -> Inserción -> Generación de Resultados. |
| **Salidas** | ID único de tarea global para el monitoreo del pipeline completo. |

### 3.1.11 Generación de SQL desde Excel con Fórmulas

Extrae la lógica de cálculo embebida en las celdas de Excel y la convierte en código de base de datos.

| | |
|---|---|
| **Entradas** | Archivo Excel con fórmulas complejas. |
| **Acción** | Extracción de fórmulas, generación de Árboles de Sintaxis Abstracta (AST) y traducción a sentencias DDL/DML compatibles con PostgreSQL. |
| **Salidas** | Diccionario de sentencias SQL generadas por cada hoja del libro de trabajo. |

### 3.1.12 Creación Dinámica de Tablas desde Esquemas o Archivos

Permite estructurar la base de datos destino basándose en la definición del esquema o en la estructura detectada en el archivo.

| | |
|---|---|
| **Entradas** | JSON Schema o archivo con tipos de datos declarados (`dtypes_str`). |
| **Acción** | Mapeo de tipos Excel/JSON a tipos SQL y ejecución de comandos `CREATE TABLE`. |
| **Salidas** | Confirmación de la creación de la tabla y metadatos del esquema resultante. |

### 3.1.13 Consulta de Estado y Resultados de Tareas

Permite a los usuarios y servicios externos conocer el progreso de cualquier operación asíncrona.

| | |
|---|---|
| **Entradas** | `task_id`. |
| **Acción** | Consulta en la capa de caché (Redis) con fallback a la persistencia (PostgreSQL/MongoDB). |
| **Salidas** | Objeto con el estado (`PENDING`, `PROCESSING`, `COMPLETED`, `FAILED`) y resultados o errores asociados. |

### 3.1.14 Monitoreo de Salud y Métricas del Sistema

Provee visibilidad técnica sobre el estado operativo de los microservicios.

| | |
|---|---|
| **Entradas** | Solicitudes a los endpoints de salud (`/healthcheck`) o métricas (`/metrics`). |
| **Acción** | Verificación de conectividad con dependencias y recolección de métricas de runtime. |
| **Salidas** | Reporte de salud y telemetría en formato compatible con Prometheus. |

## 3.2 Requerimientos no funcionales

Los requerimientos no funcionales definen las propiedades de calidad, restricciones técnicas y criterios transversales que debe cumplir S.L.O.T.H. para operar de forma segura, eficiente, escalable y resiliente. A diferencia de los requerimientos funcionales, estos no describen acciones directas del usuario, sino condiciones obligatorias sobre el comportamiento del sistema, su infraestructura, mecanismos de comunicación, rendimiento y capacidad de recuperación ante fallos.

Comprendiendo que el sistema procesa archivos con información potencialmente sensible y migra información hacia destinos privados, estos requerimientos son fundamentales para garantizar a confiabilidad de la solución, la protección de credenciales, trazabilidad de las cargas y la continuidad del procesamiento ETL.

### 3.2.1 Seguridad y privacidad

La arquitectura de S.L.O.T.H. debe implementar mecanismos de seguridad orientados a proteger las credenciales, controlar el acceso de los usuarios, preservar la integridad de los datos procesados y evitar operaciones duplicadas o no autorizadas.

**RNF-01. Cifrado de credenciales**: Las credenciales utilizadas para conectarse a bases de datos destino, servicios internos, colas de mensajería y demás recursos críticos deben protegerse mediante mecanismos de cifrado robustos, preferiblemente AES-256 o un esquema equivalente de cifrado autenticado.

Las claves de cifrado deberán gestionarse de forma separada al código fuente y a los archivos de configuración versionados.

**RNF-02. Protección de contraseñas de usuario**: Las contraseñas de los usuarios registrados en la plataforma deben almacenarse utilizando algoritmos seguros de hashing y derivación de claves, como bcrypt, Argon2 o PBKDF2. El sistema no debe permitir el almacenamiento ni la recuperación de contraseñas en texto plano.

**RNF-03. Control de acceso basado en roles:** El sistema debe aplicar un modelo de autorización basado en roles para restringir las operaciones disponibles según el perfil del usuario. Las acciones críticas, como la creación o modificación de reglas de validación, activación de esquemas, carga de archivos, consulta de resultados históricos o administración de conexiones, deberán estar limitadas a usuarios con permisos explícitos.

**RNF-04. Idempotencia en procesos de carga**: El sistema debe garantizar que la carga repetida de un mismo archivo no genere duplicidad de datos en el data warehouse. Para ello, deberá utilizar mecanismos de identificación determinística, como hashes calculados sobre el contenido del archivo, metadatos de carga

o identificadores únicos de procesamiento. Cuando el sistema detecte una carga previamente ejecutada, deberá impedir la duplicación o reutilizar el resultado del procesamiento anterior según la política definida.

**RNF-05. Trazabilidad de operaciones sensibles**: El sistema debe registrar las operaciones relevantes realizadas por los usuarios y servicios internos, incluyendo cargas de archivos, modificaciones de reglas, activación o desactivación de esquemas, errores de validación, reintentos y resultados de procesamiento. Estos registros deberán permitir identificar el usuario o componente responsable, la fecha y hora de ejecución, el recurso afectado y el resultado de la operación.

**RNF-06. Protección de datos durante la comunicación interna**: La comunicación entre servicios debe realizarse mediante canales seguros cuando el despliegue lo requiera, especialmente en ambientes distribuidos o de producción. Las llamadas entre microservicios mediante gRPC deberán poder configurarse con mecanismos de cifrado en tránsito, autenticación de servicios y validación de identidad para evitar accesos no autorizados o interceptación de datos.

### 3.2.2 Rendimiento y escalabilidad

S.L.O.T.H. debe mantener un comportamiento eficiente durante la validación, traducción y carga de archivos de hoja de cálculo, incluso cuando aumente el volumen y dimensión de los datos, el número de reglas de validación o la concurrencia de usuarios. La arquitectura debe permitir escalar sus componentes de forma independiente, evitando que un único módulo limite el rendimiento global del sistema.

**RNF-07: Procesamiento Paralelo**: El sistema debe permitir la ejecución paralela de tareas de validación, parsing y carga mediante workers configurables. La cantidad de hilos, procesos o instancias de ejecución deberá poder ajustarse de acuerdo con los recursos disponibles en el entorno de despliegue, permitiendo mejorar los tiempos de procesamiento sin modificar la lógica funcional del sistema.

**RNF-08. Escalabilidad horizontal de componentes críticos**: Los componentes responsables de validación, carga, parsing y creación de estructuras deberán poder escalar horizontalmente mediante la ejecución de múltiples instancias independientes. Esta capacidad debe permitir distribuir la carga de trabajo entre varios workers o nodos de procesamiento, especialmente en escenarios de archivos grandes o múltiples cargas concurrentes.

**RNF-09. Arquitectura desacoplada por microservicios:** El sistema debe mantener una arquitectura desacoplada basada en microservicios, de modo que cada componente pueda evolucionar, desplegarse, monitorearse y escalarse de manera independiente. Esta separación debe reducir el acoplamiento entre la interfaz web, el repositorio de reglas, el motor de validación, el sistema de parsing, el sistema de creación de tablas y los servicios de persistencia.

**RNF-10. Procesamiento asincrónico mediante cola de mensajes**: El sistema debe utilizar una cola de mensajería, como RabbitMQ, para gestionar tareas de procesamiento de forma asincrónica. Este mecanismo deberá permitir desacoplar la recepción de archivos del procesamiento interno, distribuir tareas entre workers, controlar la carga del sistema y evitar bloqueos en la interfaz de usuario durante operaciones de larga duración.

**RNF-11. Baja latencia en comunicación entre servicios core:** La comunicación entre servicios principales debe realizarse mediante mecanismos eficientes, como gRPC, con el fin de reducir la sobrecarga

de serialización y mejorar el rendimiento de las llamadas internas. Esta comunicación deberá ser especialmente eficiente entre los componentes de validación, parsing, creación de estructuras y persistencia de resultados.

**RNF-12. Crecimiento predecible del tiempo de procesamiento**: El sistema debe presentar un crecimiento predecible y preferiblemente lineal del tiempo de procesamiento ante incrementos en el volumen de filas, número de columnas o cantidad de reglas aplicadas. El diseño debe evitar cuellos de botella exponenciales en la validación, traducción o carga de datos, permitiendo estimar el comportamiento del sistema bajo distintos tamaños de entrada.

### 3.2.3 Disponibilidad y resiliencia

S.L.O.T.H. debe ser capaz de mantener la continuidad del procesamiento ante fallos parciales, interrupciones temporales o errores en componentes específicos. Dado que el sistema opera sobre procesos ETL potencialmente extensos, su arquitectura debe contemplar mecanismos de recuperación, persistencia de estado, reintentos y aislamiento de errores.

**RNF-13. Estrategia de caché dual:** El sistema debe utilizar Redis para almacenar información efímera o de acceso rápido, como estados temporales, resultados parciales o datos de coordinación entre procesos. Sin embargo, la información crítica asociada a tareas, reglas, resultados de carga y trazabilidad deberá persistir en repositorios duraderos, garantizando que una falla en la caché no comprometa la integridad del procesamiento.

**RNF-14. Reintentos automáticos ante fallos transitorios:** El sistema debe implementar mecanismos de reintento automático para tareas que fallen por causas transitorias, como errores temporales de red, indisponibilidad momentánea de servicios, fallos de conexión con la base de datos o interrupciones en la cola de mensajes. La política de reintentos deberá definir un número máximo de intentos y tiempos de espera progresivos para evitar ciclos infinitos de ejecución.

**RNF-15. Recuperación ante interrupciones de procesos ETL:** Cuando un proceso ETL sea interrumpido, el sistema debe permitir identificar el punto de falla y conservar suficiente información para decidir si la tarea puede reanudarse, reintentarse o marcarse como fallida. Esta capacidad es necesaria para evitar reprocesamientos innecesarios y reducir la pérdida de trabajo en archivos de gran tamaño.

### 3.2.4 Mantenibilidad y extensibilidad

El sistema debe facilitar su evolución técnica, permitiendo incorporar nuevas reglas, conectores, formatos, motores de base de datos y mecanismos de procesamiento sin rediseñar la arquitectura completa.

**RNF-16. Separación de responsabilidades**: Cada componente del sistema debe tener responsabilidades claramente delimitadas. La interfaz web no debe contener lógica de validación profunda, el motor de validación no debe encargarse de la administración visual de reglas, y el sistema de parsing debe mantenerse separado de la carga de datos. Esta separación mejora la mantenibilidad, las pruebas y la evolución independiente de los módulos.

**RNF-17. Extensibilidad de reglas de validación**: El sistema debe permitir incorporar nuevos tipos de reglas de validación sin modificar de forma significativa los componentes existentes. La representación

basada en esquemas debe facilitar la adición de restricciones futuras sobre formatos, dependencias, dominios permitidos o validaciones compuestas.

**RNF-18. Extensibilidad del sistema de parsing**: El sistema de parsing debe permitir la incorporación progresiva de nuevas funciones, operadores y patrones de fórmulas provenientes de hojas de cálculo. Cuando una fórmula no pueda traducirse a SQL, el sistema deberá reportarla de forma explícita y evitar traducciones parciales que puedan comprometer la integridad de los cálculos.

# 4. Arquitectura

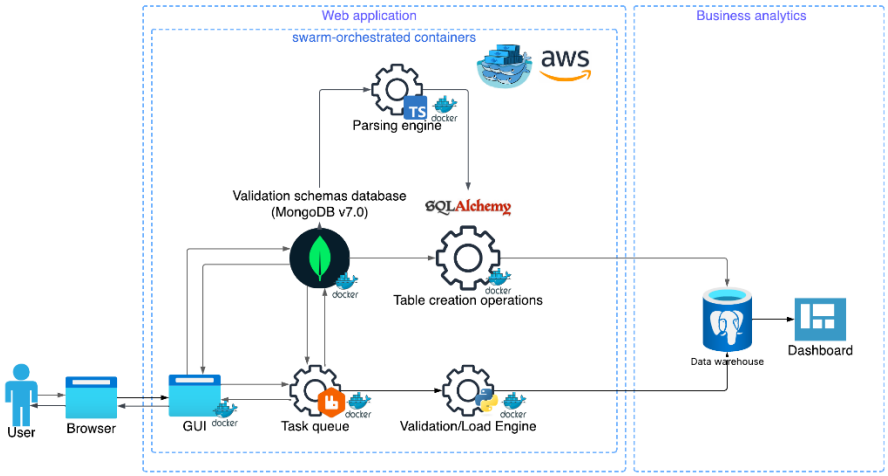



*Imagen 1: diagrama de arquitectura tecnológica (resumen).*

## 4.1 Propósito de la arquitectura

El objetivo de S.L.O.T.H es simplificar para el usuario el acceso a un flujo ETL distribuido, trazable y escalable para la validación, carga y migración de lógica de negocio contenida en hojas de cálculo. A diferencia de una arquitectura ETL tradicional, donde las reglas de validación y transformación suelen encontrarse embebidas directamente en scripts o archivos manipulados por usuarios finales, nuestro sistema propone una separación explícita entre la lógica de negocio, el procesamiento de datos y los mecanismos de persistencia.

Desde el punto de vista del dominio, el sistema busca resolver tres problemas principales. Primero, la dispersión de reglas de negocio en hojas de cálculo, fórmulas y código fuente. Segundo, la dificultad de auditar, versionar y modificar dichas reglas sin intervención técnica directa. Tercero, la limitada escalabilidad de los procesos ETL secuenciales cuando se enfrentan a archivos tabulares de gran volumen o alta dimensionalidad. En consecuencia, la arquitectura se diseña para permitir la gestión de usuarios, proyectos y esquemas de validación; el procesamiento asincrónico de archivos .xlsx, .xls y .csv; la traducción de fórmulas de Excel a SQL; y la trazabilidad completa del ciclo de vida de tareas, reglas y resultados de carga.

Estos objetivos arquitectónicos se implementan mediante una estructura distribuida de microservicios con responsabilidades acotadas, contratos explícitos y mecanismos de integración tanto sincrónicos como

asincrónicos. El diseño combina dos patrones de interacción. Por un lado, utiliza orquestación sincrónica mediante API's para operaciones administrativas, consultas de estado, gestión de usuarios, proyectos y reglas. Por otro lado, emplea asincronía mediante colas de mensajes para operaciones pesadas o de larga duración, como validación, inserción y procesamiento de resultados. Esta separación permite que las operaciones de usuario no queden bloqueadas por procesos ETL intensivos y que los servicios internos puedan escalar de acuerdo con la demanda.

## 4.2 Estilo arquitectónico

S.L.O.T.H. adopta un estilo de microservicios colaborativos, en el que cada servicio encapsula una capacidad funcional o técnica claramente delimitada. Esta decisión responde a la necesidad de aislar responsabilidades, permitir despliegues independientes y facilitar la evolución progresiva de componentes como el parser, el motor de validación o el servicio de persistencia operacional sin necesidad de redespliegues masivos o modificaciones generalizadas. De esta manera, la arquitectura se organiza alrededor de cinco bloques principales:

1. **Web App:** Capa de presentación del sistema, implementada con Nuxt, encargada de ofrecer la experiencia de usuario para la gestión de proyectos, esquemas, archivos y consultas de estado.
2. **Core API:** Fachada principal de negocio, implementada con FastAPI, responsable de autenticar usuarios, validar permisos, registrar tareas, aplicar idempotencia, publicar mensajes en RabbitMQ y coordinar operaciones sincrónicas con otros servicios.
3. **Typechecking Service:** Subsistema asincrónico compuesto por workers especializados en validación, inserción y consolidación de resultados. Su propósito es ejecutar procesos de alto costo computacional de forma desacoplada respecto a las solicitudes de usuario.
4. **Parsing Subsystem:** Conjunto de servicios especializados en lectura de archivos Excel, análisis de fórmulas, generación de expresiones SQL y construcción de sentencias ejecutables. Este subsistema permite migrar lógica de negocio desde celdas de hojas de cálculo hacia el motor de base de datos destino.
5. **Database Service:** Servicio gRPC que funciona como frontera unificada para operaciones sobre Redis y MongoDB, incluyendo estados operacionales, caché, esquemas JSON, versiones de reglas y tareas distribuidas

Este estilo permite combinar escalabilidad, modularidad y trazabilidad. Además, reduce el acoplamiento entre componentes que evolucionan a ritmos distintos: por ejemplo, el parser puede incorporar nuevas funciones de Excel sin afectar a la interfaz web o al typechecking.

## 4.3 Principios de diseño

A lo largo del proyecto fueron elegidos un conjunto de principios para orientar la organización interna del sistema y la toma de decisiones tecnológicas, que serán listados a continuación.

### 4.3.1 Responsabilidad única por servicio

Cada servicio debe encapsular una responsabilidad principal. La Web App no contiene lógica profunda de validación; el Core API no ejecuta procesamiento intensivo de archivos; los workers no administran usuarios ni permisos; y el subsistema de parsing no decide políticas de autorización. Esta separación evita

la concentración de responsabilidades en un único componente y facilita pruebas, mantenimiento y evolución.

### 4.3.2 Contratos explícitos entre servicios

Las comunicaciones internas críticas utilizan contratos definidos mediante gRPC y Protocol Buffers. Esta decisión permite establecer interfaces tipadas, versionables y eficientes entre servicios, reduciendo ambigüedades en el intercambio de datos. En particular, los servicios de parsing y persistencia operacional se benefician de contratos estrictos, dado que procesan estructuras complejas como AST, expresiones SQL, esquemas JSON y estados de tareas.

### 4.3.3 Idempotencia operativa

Los procesos de carga deben evitar duplicidades cuando un usuario envía el mismo archivo más de una vez, cuando ocurre un reintento o cuando un worker reprocesa un mensaje. Para ello, la arquitectura incorpora una clave de idempotencia construida a partir del contenido del archivo y su contexto de ejecución. Esta política permite identificar tareas duplicadas y prevenir inserciones repetidas en el data warehouse.

### 4.3.4 Externalización del estado

Los workers se diseñan como unidades de ejecución sin estado persistente propio. El progreso de tareas, resultados intermedios, reglas activas y estados de procesamiento se almacenan fuera de los workers, principalmente en PostgreSQL, Redis y MongoDB. Esta decisión permite reiniciar workers, escalar instancias o recuperar tareas sin perder trazabilidad del proceso.

### 4.3.5 Caché como acelerador, no como fuente de datos determinante

Redis se utiliza para mejorar tiempos de acceso a información operacional, pero no debe ser la única fuente del sistema. La información crítica asociada a reglas, versiones, tareas y resultados debe persistirse en repositorios duraderos como MongoDB o PostgreSQL. Esta estrategia permite tolerar fallos de caché sin comprometer la integridad del flujo ETL.

### 4.3.6 Trazabilidad extremo a extremo

Cada proceso de carga debe ser monitoreable desde la solicitud inicial hasta su cierre. Para ello, la arquitectura crea y transmite identificadores de tarea y contexto de ejecución a través de HTTP, AMQP y gRPC. Esta trazabilidad permite reconstruir el flujo completo de una operación, diagnosticar fallos y auditar qué reglas fueron aplicadas sobre un archivo específico.

## 4.4 Vista general de componentes

La arquitectura de S.L.O.T.H. está organizada como una cadena de colaboración entre el frontend, las API’s, los servicios de mensajería, los workers, nodos del parser y servicios de estado. El usuario interactúa con la Web App, la cual consume los endpoints expuestos por el Core API. Este último actúa como punto de control para autenticación, autorización, registro de tareas y coordinación de flujos. Luego las operaciones pesadas se delegan a RabbitMQ y son procesadas por workers especializados.

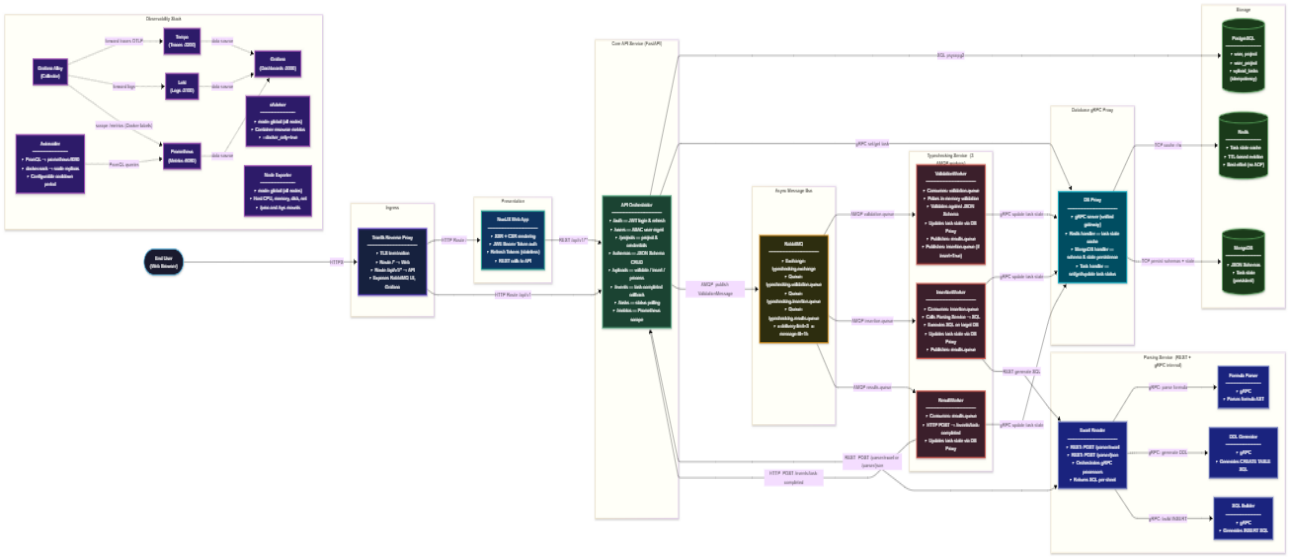


*Imagen 2: diagrama de arquitectura.*

La vista general permite observar cómo el sistema separa las responsabilidades de interacción, control, procesamiento, parsing y persistencia. Esta separación es relevante porque S.L.O.T.H. no ejecuta todo el flujo ETL dentro de una única aplicación monolítica, sino que distribuye el trabajo entre servicios especializados.

La Web App centraliza la interacción con los usuarios. Desde esta capa se gestionan proyectos, esquemas de validación, archivos cargados y consultas de estado. También puede incluir vistas tabulares o JSON del contenido procesado y funcionalidades avanzadas como un query builder asistido.

El Core API funciona como la frontera principal de negocio. Sus responsabilidades incluyen autenticación, control de acceso, validación de permisos, creación de tareas, publicación de mensajes y consolidación de respuestas. Además, es el componente que mantiene la coherencia de alto nivel del sistema, ya que registra formalmente las tareas y sus estados principales.

RabbitMQ permite desacoplar temporalmente la recepción de archivos y la ejecución del procesamiento. Esta cola de mensajes evita que la solicitud HTTP del usuario permanezca bloqueada mientras se validan o insertan grandes volúmenes de datos.

Los Typechecking Workers ejecutan el procesamiento asincrónico. Estos workers validan archivos contra JSON Schema, solicitan instrucciones SQL de inserción cuando corresponde y publican resultados para cerrar el ciclo de la tarea.

El Parsing Subsystem se encarga de analizar las hojas de cálculo, detectar fórmulas, construir representaciones intermedias y traducir lógica de negocio a SQL. Su rol es fundamental para migrar lógica que originalmente estaba encapsulada en celdas hacia el data warehouse.

El Database Service unifica el acceso a Redis y MongoDB para estados operacionales, caché, esquemas y versionamiento de reglas. Su existencia evita que cada microservicio implemente clientes y políticas de persistencia propias.

## 4.5 Vista lógica de la API central

El Core API no se concibe como una capa plana de endpoints, sino como una aplicación organizada internamente por responsabilidades. Su diseño separa transporte, reglas de negocio, acceso a datos y servicios de integración.

La capa de routes expone la superficie HTTP de la aplicación. En esta capa se ubican endpoints asociados a autenticación, usuarios, proyectos, esquemas, cargas, tareas, eventos, caché y estado de salud. Su función principal es recibir solicitudes, validar entradas básicas y delegar la ejecución a servicios de dominio.

La capa de dependencies centraliza la inyección de dependencias necesarias para resolver cada caso de uso. Entre estas dependencias se encuentran la sesión SQL, el cliente gRPC hacia el Database Service, el publicador de RabbitMQ y el usuario autenticado. Esta capa evita que la inicialización de recursos se replique en cada endpoint.

La capa de services contiene la lógica de negocio. Allí se implementan políticas como idempotencia, validación de permisos, gestión de usuarios, administración de proyectos, procesamiento de uploads y coordinación de esquemas. Esta capa permite reutilizar reglas entre endpoints y mantener la lógica crítica fuera de los controladores HTTP.

La capa de repositories abstrae el acceso a PostgreSQL para entidades transaccionales, como usuarios, proyectos, relaciones usuario-proyecto y tareas de carga. Esta separación permite modificar detalles de persistencia sin alterar directamente la lógica de negocio.

Finalmente, la capa de models/schemas define las entidades SQLAlchemy y los contratos Pydantic utilizados para validar entradas, estructurar salidas y representar entidades internas.

Esta organización interna favorece la mantenibilidad del Core API, ya que impide que los endpoints acumulen lógica compleja. También permite aplicar de manera uniforme políticas de seguridad, autorización, validación y trazabilidad.

## 4.6 Microservicios y responsabilidades

### 4.6.1 Core API service

El Core API Service es el punto de entrada principal para las operaciones de negocio. Su función es coordinar todo el procesamiento ETL. Este servicio recibe solicitudes desde la aplicación web, valida la identidad del usuario, verifica permisos sobre proyectos y recursos, registra tareas y decide si una operación debe resolverse de forma sincrónica o delegarse al pipeline asincrónico.

Entre sus responsabilidades principales se encuentran la autenticación de usuarios, la gestión de proyectos, la aplicación de políticas RBAC o ABAC, el registro de tareas con clave de idempotencia, la publicación de mensajes hacia RabbitMQ, la consulta de estados y la recepción de callbacks de cierre provenientes del ResultWorker.

Arquitectónicamente, el Core API mantiene la consistencia de alto nivel mediante PostgreSQL, especialmente a través de la entidad upload_tasks. Al mismo tiempo, utiliza el Database Service como capa

operacional distribuida para consultar o actualizar estados intermedios, reglas activas y datos almacenados en Redis o MongoDB.

### 4.6.2 Database Service

El Database Service actúa como un proxy gRPC para operaciones de estado y persistencia no relacional. Su objetivo es ofrecer una frontera común para acceder a Redis y MongoDB sin que cada servicio de la plataforma deba implementar su propia lógica de conexión, serialización, fallback y consulta.

Este servicio administra operaciones de caché en Redis, incluyendo lectura, escritura, limpieza y consulta de llaves temporales. También gestiona operaciones sobre MongoDB relacionadas con JSON Schemas, versionamiento de reglas, búsqueda de esquemas, reversión de versiones y almacenamiento de estados operacionales de tareas.

El valor arquitectónico de este componente está en la homogeneización de contratos. Al centralizar el acceso a estado distribuido, se reduce la duplicación de lógica entre API, workers y servicios auxiliares. Además, se pueden aplicar políticas comunes de fallback entre caché y persistencia duradera.

### 4.6.3 Typechecking Service

El Typechecking Service representa el subsistema de procesamiento asincrónico orientado a throughput y resiliencia. Está compuesto por workers que consumen mensajes de RabbitMQ y ejecutan etapas específicas del flujo ETL:

- El **ValidationWorker** valida los archivos contra el JSON Schema activo asociado a la entidad de destino. Su responsabilidad es identificar registros válidos e inválidos, generar resultados de validación y decidir si el flujo puede continuar hacia inserción.
- El **InsertionWorker** toma los datos validados, solicita al subsistema de parsing las instrucciones SQL necesarias y ejecuta la inserción contra la base de datos destino del proyecto. Este worker desacopla la carga efectiva de datos de la solicitud inicial del usuario.
- El **ResultWorker** consolida el resultado del proceso, publica el estado final y notifica al Core API mediante un callback. Este cierre permite actualizar la tarea en PostgreSQL y dejar el proceso disponible para consulta histórica.

El patrón de ejecución de estos workers incluye consumo por colas, confirmación explícita de mensajes, reintentos en fallos de infraestructura, envío de tareas a colas de error y omisión idempotente cuando una tarea ya fue cerrada o se encuentra en progreso.

### 4.6.4 Parsing Subsystem

El Parsing Subsystem es el componente especializado en migrar lógica de negocio desde hojas de cálculo hacia SQL. Se diseña como un pipeline por etapas para separar problemas sintácticos, semánticos y estructurales.

1. El **Excel Reader** funciona como orquestador del pipeline. Recibe archivos o estructuras de entrada, identifica contenido relevante, coordina los servicios internos y retorna sentencias o fragmentos SQL según el caso de uso.

2. El **Formula Parser**, implementado como servicio gRPC en Node.js, transforma fórmulas de Excel en tokens y árboles de sintaxis abstracta. Esta representación permite analizar la estructura lógica de las fórmulas de forma más robusta que mediante tratamiento textual directo.
3. El **DDL Generator**, implementado en Python, transforma representaciones intermedias en expresiones SQL equivalentes cuando la fórmula o función tiene soporte en el motor destino.
4. El **SQL Builder** ensambla las expresiones generadas y ordena las operaciones según dependencias entre columnas o celdas. Esto permite producir SQL ejecutable en un orden consistente, evitando que una columna calculada dependa de otra que todavía no ha sido creada o poblada.

Este diseño modular permite ampliar progresivamente el soporte de funciones de Excel sin reescribir todo el subsistema. También permite reportar fórmulas no soportadas de forma explícita, evitando traducciones parciales que puedan comprometer la integridad de los cálculos.

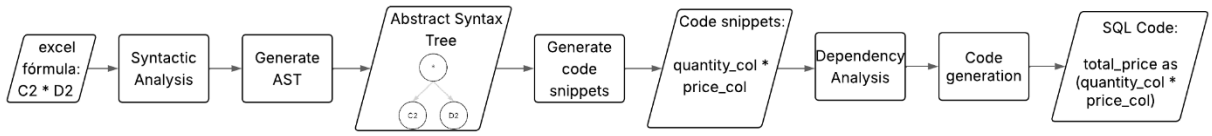



*Imagen 3: diagrama de flujo de parsing (ejemplo).*

### 4.6.5 Web App

La Web App constituye la capa de presentación y experiencia de usuario. Su función es abstraer la complejidad técnica del sistema y permitir que usuarios autorizados interactúen con proyectos, esquemas, archivos, tareas y resultados sin manipular directamente servicios internos.

Desde esta capa, el usuario puede gestionar visualmente reglas de validación, cargar archivos, consultar el estado de tareas y revisar resultados. También puede acceder a funcionalidades avanzadas, como la visualización tabular o JSON de archivos y un query builder con validación de columnas y ejecución segura de consultas SELECT.

Aunque la Web App puede incluir endpoints server-side para funcionalidades específicas, no debe asumir responsabilidades centrales de negocio. Las decisiones críticas de autorización, idempotencia, publicación de tareas y persistencia deben permanecer en el Core API.

## 4.7 Interacciones y protocolos

La arquitectura utiliza tres protocolos principales, seleccionados de acuerdo con el tipo de interacción que debe resolverse.

**HTTP/REST:** se utiliza en la frontera externa del sistema, especialmente en la comunicación entre la Web App y el Core API. También se usa para integraciones donde la interoperabilidad y simplicidad son más importantes que el rendimiento estricto, como llamadas entre el Core API y el Excel Reader para ciertos casos de parseo o creación de tablas. Además, el callback del ResultWorker hacia el Core API puede exponerse como endpoint HTTP, por ejemplo /events/task-completed.

**gRPC:** se utiliza para comunicación interna de alto rendimiento entre servicios de la plataforma. Este protocolo resulta adecuado cuando se requieren contratos tipados, baja sobrecarga de serialización y

llamadas frecuentes entre servicios. En S.L.O.T.H utilizamos gRPC entre el Excel Reader y servicios especializados como Formula Parser, DDL Generator y SQL Builder; también entre el Core API, los workers y el Database Service.

**AMQP mediante RabbitMQ:** se utiliza para coordinar tareas asincrónicas de larga duración. Las colas de validación, inserción y resultados permiten separar temporalmente la recepción de una solicitud y su procesamiento efectivo. Esto habilita reintentos, distribución de carga y procesamiento por múltiples workers.

La coexistencia de estos protocolos responde a una decisión arquitectónica intencional: REST para la frontera externa, gRPC para contratos internos estrictos y AMQP para elasticidad del procesamiento ETL.

## 4.8 Vista de flujos críticos de negocio

### 4.8.1 Gestión de reglas de validación

El flujo de gestión de reglas inicia cuando un usuario autorizado accede a la aplicación web y crea, modifica o consulta un esquema de validación. La solicitud es enviada al Core API, donde se validan permisos, consistencia del esquema y contexto del proyecto. Posteriormente, el Core API invoca al Database Service para almacenar o actualizar el JSON Schema correspondiente.

MongoDB conserva la versión activa del esquema y el historial de versiones o releases. Esta separación permite que los workers y el API consulten siempre la versión activa, mientras que las versiones anteriores permanecen disponibles para auditoría, trazabilidad o reversión.

El resultado arquitectónico de este flujo es el desacoplamiento entre reglas de negocio y código fuente. Las reglas pueden evolucionar sin necesidad de redesplegar el backend, lo que reduce dependencia técnica y mejora la gobernanza del proceso ETL.

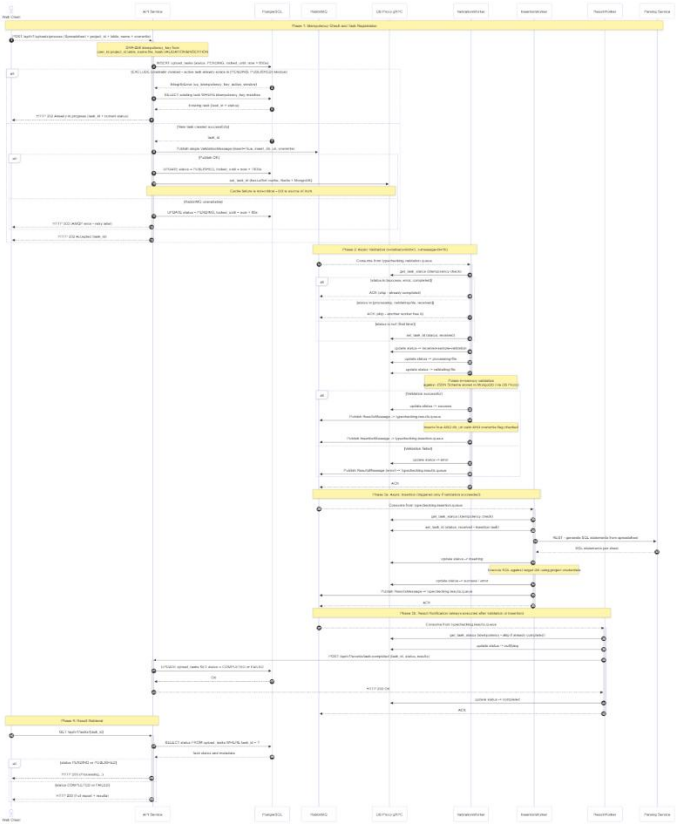


*Imagen 4: Secuencia de validación, inserción y resultados.*

### 4.8.2 Validación asincrónica de carga

El flujo de validación inicia cuando el cliente envía un archivo a un endpoint como /uploads/validate o /uploads/process. El Core API calcula un hash del archivo, genera una clave de idempotencia, registra la tarea en PostgreSQL y publica un mensaje en la cola de validación.

El ValidationWorker consume el mensaje, consulta el esquema activo asociado a la entidad destino y valida el contenido del archivo. Los registros que cumplen las reglas son marcados como válidos; los que incumplen una o más restricciones son reportados como inválidos. El resultado se publica para continuar el flujo o para consolidar el cierre de la tarea.

Este flujo permite que la carga no bloquee al usuario durante la validación. Además, el uso de task_id permite consultar el estado de la operación y recuperar el resultado de procesos de larga duración. Corresponde a la primera y segunda sección del diagrama de secuencia de la *imagen 2.*

### 4.8.3 Inserción asincrónica

El flujo de inserción se ejecuta cuando una tarea validada debe cargar registros en la base de datos destino. El InsertionWorker consume el mensaje correspondiente, solicita al Excel Reader o al subsistema de parsing el SQL de inserción necesario y ejecuta las sentencias contra el PostgreSQL destino del proyecto.

Una vez finalizada la inserción, el worker publica el resultado en la cola correspondiente. Posteriormente, el ResultWorker consolida el estado final y notifica al Core API.

Este flujo mantiene desacoplada la inserción respecto a la solicitud inicial. Además, habilita políticas de reintento, manejo de errores, control por estado e idempotencia ante duplicados. Corresponde a la tercera sección del diagrama de secuencia de la *imagen 4.*

### 4.8.4 Migración de fórmulas a SQL

El flujo de migración de fórmulas inicia cuando el Excel Reader recibe un archivo o estructura que contiene fórmulas. El Formula Parser analiza las fórmulas y construye representaciones intermedias como tokens y AST. Luego, el DDL Generator transforma esas representaciones en expresiones SQL equivalentes. Finalmente, el SQL Builder ordena dependencias y construye sentencias SQL ejecutables.

Este flujo permite que la lógica que antes estaba embebida en celdas sea migrada al motor SQL. Como consecuencia, los cálculos dejan de depender exclusivamente de archivos cliente y pasan a ejecutarse en un entorno centralizado, auditable y reproducible.

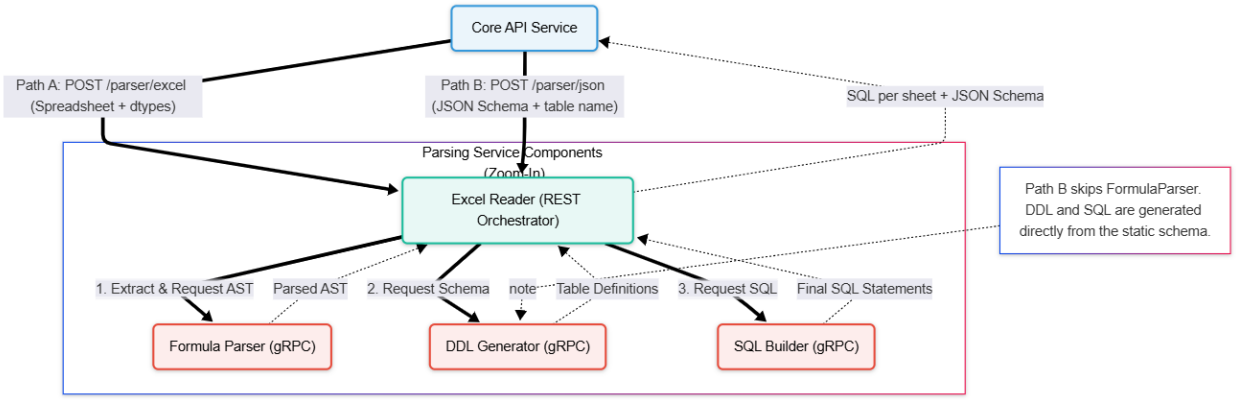



*Imagen 5: Diagrama de componentes del fórmula parser.*

## 4.9 Estados y consistencia

Como sistema, S.L.O.T.H maneja diferentes tipos de estado, cada uno con una función arquitectónica específica, el transaccional y el operacional:

El **estado transaccional** de negocio se almacena en PostgreSQL. Incluye usuarios, proyectos, relaciones entre usuarios y proyectos, y tareas base de carga. Este estado representa la fuente formal para entidades principales del sistema y para los hitos de cierre del procesamiento.

El **estado operacional** distribuido se maneja mediante Redis y MongoDB a través del Database Service. Redis se utiliza para datos temporales, cacheo de estados y acceso rápido a información de ejecución. MongoDB se utiliza para almacenar esquemas activos, historial de versiones y respaldo persistente de estados relevantes.

La política de consistencia del sistema privilegia disponibilidad, recuperación y trazabilidad en operaciones largas, en lugar de intentar imponer una transacción global distribuida entre todos los servicios. Esto implica que algunos estados intermedios pueden presentar consistencia eventual, especialmente durante el procesamiento asincrónico. Sin embargo, los hitos de cierre, como tarea completada, fallida o parcialmente procesada, deben consolidarse mediante el Core API y PostgreSQL.

Esta estrategia resulta adecuada para procesos ETL distribuidos, donde una transacción global podría aumentar el acoplamiento y reducir la resiliencia del sistema.

## 4.10 Modelos de datos arquitectónicamente relevantes

El modelado de datos combina estructuras relacionales y documentales, respondiendo a la naturaleza híbrida del sistema: algunas entidades requieren integridad transaccional estricta, mientras que otras necesitan flexibilidad estructural y versionamiento.

### 4.10.1 Entidades relacionales

Las entidades relacionales principales son user, project, user_project y upload_tasks.

La entidad user representa los usuarios registrados en la plataforma. La entidad project agrupa recursos, configuraciones y contexto de trabajo. La relación user_project define la membresía y autorización de usuarios dentro de un proyecto, permitiendo aplicar restricciones de acceso contextual. La entidad upload_tasks registra los procesos de carga, su estado, su clave de idempotencia y metadatos necesarios para controlar duplicados o ventanas activas de procesamiento.

Particularmente, upload_tasks incorpora campos como idempotency_key y locked_until, lo que permite evitar que una misma carga se procese varias veces mientras se encuentra en estados activos como ‘PENDING’ o ‘PUBLISHED’.

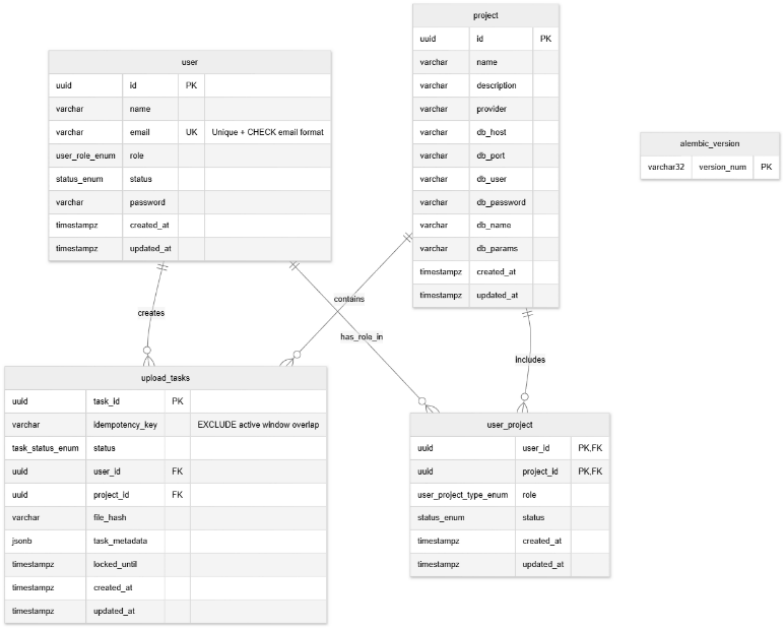



*Imagen 6: Modelo relacional del core API.*

### 4.10.2 Estructuras documentales

Las estructuras documentales se concentran principalmente en MongoDB. Entre ellas se encuentran active_schema y schemas_releases, utilizadas para gestionar esquemas activos y versiones históricas de reglas de validación.

El uso de documentos resulta adecuado para representar JSON Schemas, ya que estos pueden variar en estructura, cantidad de campos, tipos de restricciones y complejidad. Además, MongoDB facilita conservar versiones sucesivas de un esquema sin forzar una estructura relacional rígida.

También se almacenan estados de tareas por task_id o import_name, lo que permite hacer seguimiento operacional del pipeline y recuperar información intermedia en caso de fallos.

# 5. Infraestructura y despliegue

Esta sección corresponde a la vista física y operacional del sistema. Mientras la sección previa de arquitectura describe cómo se organizan y colaboran los componentes lógicos, en este apartado se describe dónde se ejecutan los componentes, cómo se aprovisionan los recursos, cómo se despliegan los servicios, qué mecanismos permiten su comunicación y qué herramientas soportan su operación continua.

## 5.1 Propósito de la infraestructura

La infraestructura de S.L.O.T.H. está diseñada para sostener la operación de una plataforma ETL distribuida basada en microservicios, procesamiento asincrónico, persistencia híbrida, comunicación interna entre servicios y mecanismos de observabilidad, por lo que no puede limitarse a alojar contenedores, sino que

debe proporcionar un entorno operativo reproducible, escalable, monitoreable y controlado para ejecutar procesos de validación, parsing, inserción y seguimiento de operaciones.

La infraestructura debe cumplir cuatro objetivos principales: Primero, permitir que el sistema pueda desplegarse de forma consistente en distintos ambientes. Segundo, soportar la ejecución distribuida de servicios y workers mediante un clúster de contenedores. Tercero, proporcionar mecanismos de persistencia, mensajería, caché y almacenamiento compartido necesarios para el funcionamiento de la plataforma. Cuarto, habilitar monitoreo, trazabilidad y entrega continua para reducir la dependencia de operaciones manuales.

Con la consecución de estos objetivos, la infraestructura actúa como el soporte técnico que permite materializar los atributos de calidad definidos en el documento: escalabilidad, disponibilidad, resiliencia, mantenibilidad, observabilidad y seguridad operativa.

## 5.2 Enfoque general

S.L.O.T.H. adopta un enfoque de IaC (Infraestructura como código) en el cual los recursos, configuraciones y procesos de despliegue se definen mediante archivos versionados. Esta decisión busca evitar configuraciones manuales difíciles de reproducir y permitir que los ambientes puedan reconstruirse de manera controlada a partir del repositorio del proyecto. Esta metodología se apoya en cuatro herramientas principales:

1. **Terraform**: Para aprovisionar los recursos base en AWS, incluyendo red, instancias, grupos de seguridad, perfiles IAM, almacenamiento compartido y llaves de acceso.
2. **Ansible**: Para configurar las instancias aprovisionadas, instalar dependencias del sistema operativo, preparar Docker, montar almacenamiento compartido e inicializar el clúster de Docker Swarm.
3. **Docker Swarm**: Utilizado como orquestador de contenedores para desplegar, replicar, actualizar y operar los servicios de aplicación, infraestructura interna y observabilidad.
4. **GitHub Actions**: Para automatizar procesos de integración continua, construcción de imágenes, publicación de artefactos, actualización de stacks y despliegue hacia los ambientes configurados.

Esta separación de responsabilidades permite que cada herramienta intervenga en una etapa específica del ciclo de vida de la infraestructura. Terraform define la topología base, Ansible convierte las instancias en nodos funcionales, Docker Swarm administra las cargas de trabajo, y GitHub Actions conecta los cambios del repositorio con el pipeline de entrega continua.

La ventaja principal de este enfoque es la reproducibilidad, ya que el ambiente de despliegue no depende de configuraciones aplicadas manualmente sobre los servidores, sino de definiciones declarativas que pueden revisarse, auditarse, versionarse y ejecutarse nuevamente cuando sea necesario.

## 5.3 Infraestructura física

### 5.3.1 Aprovisionamiento en AWS

La infraestructura base de S.L.O.T.H. se aprovisiona sobre Amazon Web Service (de aquí en adelante AWS) mediante el módulo Terraform (véase en el repositorio la dirección *iac/terraform/module/swarm-cluster*). Este módulo define los recursos mínimos necesarios para ejecutar un clúster orientado a Docker

Swarm y permite parametrizar aspectos propios de cada ambiente; como cantidad de nodos, tipo de instancia, reglas de acceso, almacenamiento compartido y configuración de red.

La infraestructura base incluye una red virtual privada (de aquí en adelante VPC por sus siglas en inglés) dedicada, que aísla los recursos del sistema dentro de un espacio de red propio. Dentro de esta VPC se definen subredes públicas distribuidas en zonas de disponibilidad configurables, lo que permite ubicar nodos del clúster en distintas zonas y mejorar la tolerancia ante fallos de infraestructura.

También se configura un Internet Gateway y tablas de enrutamiento para habilitar la conectividad externa controlada. Esta conectividad es necesaria para exponer la aplicación web, permitir el acceso HTTP/HTTPS a través de Traefik y facilitar operaciones administrativas autorizadas.

Los Security Groups definen la superficie de red permitida, restringiendo el tráfico entrante a los puertos requeridos para SSH, HTTP, HTTPS, comunicación interna de Docker Swarm, overlay networking y acceso a almacenamiento compartido mediante NFS cuando EFS se encuentra habilitado.

La capa de cómputo se compone de instancias EC2 diferenciadas en nodos manager y worker. Los managers mantienen el estado del clúster y ejecutan operaciones de administración, mientras que los workers alojan réplicas de servicios de aplicación y procesamiento. Esta separación permite distribuir responsabilidades y escalar capacidad de ejecución sin comprometer el plano de control.

La infraestructura también contempla un IAM Instance Profile asociado a las instancias, con integración a AWS Systems Manager. Esto permite capacidades de administración gestionada y reduce la dependencia exclusiva del acceso SSH manual.

Cuando se requiere almacenamiento compartido, el módulo puede aprovisionar Elastic File System (EFS), montado sobre /mnt/shared. Este recurso permite que varios nodos accedan a rutas comunes, útiles para certificados, artefactos de despliegue o configuraciones persistentes.

Para una visualización general de la infraestructura, diríjase a la *imagen 5*.

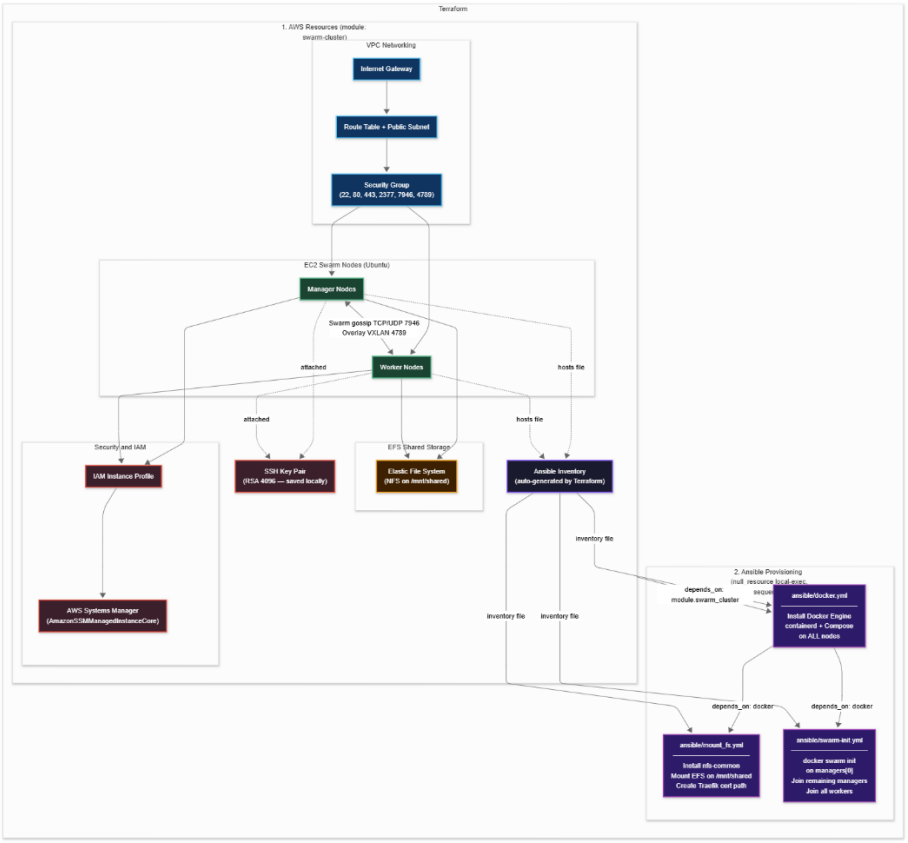



*Imagen 7: Diagrama de infraestructura de despliegue.*

### 5.3.2 Roles de nodos del clúster

El clúster se organiza en dos tipos de nodos: manager nodes y worker nodes.

Los manager nodes mantienen el estado del clúster de Docker Swarm, gestionan la planificación de servicios, coordinan actualizaciones y reciben operaciones administrativas, y debido a su rol en el plano de control, estos nodos deben tratarse como componentes críticos de la infraestructura. En ellos pueden ubicarse servicios que requieran mayor control operativo, como entrada de tráfico, observabilidad o tareas de administración.

Los worker nodes ejecutan las cargas de trabajo de aplicación y procesamiento. En estos nodos pueden desplegarse réplicas de servicios como API, workers de validación, workers de inserción, servicios de parsing y otros componentes que soporten escalamiento horizontal. Su función principal es absorber carga de ejecución sin concentrar responsabilidades de gobierno del clúster.

Esta separación permite que el sistema escale de forma más ordenada. Si aumenta la cantidad de archivos a procesar, pueden incorporarse más workers o aumentar réplicas de servicios específicos sin alterar el rol de los managers. Asimismo, facilita aplicar restricciones de despliegue, por ejemplo, ubicando ciertos servicios únicamente en managers o distribuyendo workers según disponibilidad de recursos.

### 5.3.3 Almacenamiento compartido

La infraestructura puede habilitar un sistema de almacenamiento compartido mediante **Amazon EFS**, montado en la ruta /mnt/shared. Este almacenamiento permite que los nodos del clúster accedan a archivos comunes de manera uniforme, independientemente de dónde se ejecute un contenedor determinado.

El almacenamiento compartido es útil para recursos que deben sobrevivir a reinicios, redeploys o reubicación de contenedores entre nodos. Entre estos recursos se encuentran certificados TLS utilizados por Traefik, artefactos compartidos de despliegue, archivos de configuración y directorios persistentes que no deben depender exclusivamente del disco local de una instancia.

El uso de EFS reduce el acoplamiento entre servicios y nodos físicos. En un clúster donde Docker Swarm puede reubicar tareas por fallos, actualizaciones o cambios de capacidad, depender del disco local de un nodo podría generar inconsistencias. Al utilizar almacenamiento compartido, ciertos recursos operativos permanecen disponibles para el conjunto del clúster.

No obstante, el almacenamiento compartido debe utilizarse de forma controlada. Los datos transaccionales principales del sistema deben permanecer en sus servicios de persistencia correspondientes, como PostgreSQL, MongoDB o Redis, y no delegarse indiscriminadamente al filesystem compartido.

## 5.4 Aprovisionamiento e inicialización del clúster

### 5.4.1 Aprovisionamiento con terraform

Terraform se utiliza para declarar y crear la infraestructura base de cada ambiente. Los directorios *iac/terraform/environments/staging* e *iac/terraform/environments/production* invocan el módulo swarm-cluster con variables específicas para cada entorno.

El flujo de aprovisionamiento incluye la creación de la red base, subredes, rutas, grupos de seguridad, llaves SSH, instancias EC2, perfiles IAM y almacenamiento EFS cuando se encuentra habilitado. Además, Terraform genera un inventario de Ansible a partir de las direcciones IP de las instancias creadas, lo que permite encadenar el aprovisionamiento con la configuración posterior del sistema operativo.

Un aspecto relevante del diseño es que Terraform no se limita a crear recursos en AWS, sino que también puede iniciar el proceso de bootstrap mediante recursos como null_resource y ejecuciones locales. Esto permite que un mismo levantamiento de infraestructura deje preparado el entorno para recibir cargas de trabajo, reduciendo pasos manuales y disminuyendo el riesgo de inconsistencias entre ambientes.

En general, esta metodología refuerza la reproducibilidad y auditabilidad de la infraestructura, pues los cambios en red, nodos, almacenamiento o seguridad pueden revisarse como cambios de código antes de ser aplicados al ambiente correspondiente.

### 5.4.2 Configuración del sistema utilizando Ansible

Una vez creadas las instancias, Ansible se encarga de preparar los nodos para operar dentro del clúster, para lo que se ejecuta tres playbooks principales: docker.yml, mount_fs.yml y swarm-init.yml.

El playbook docker.yml instala las dependencias base del sistema operativo, configura el repositorio oficial de Docker, instala docker-ce, containerd y el plugin de Docker Compose. También habilita los servicios

necesarios para arranque automático y agrega el usuario operativo al grupo docker, permitiendo que las operaciones de despliegue puedan ejecutarse sin intervención manual sobre cada nodo.

El playbook mount_fs.yml instala el cliente NFS, crea el punto de montaje /mnt/shared, monta el sistema EFS de forma persistente y prepara rutas utilizadas por servicios como Traefik. Este paso garantiza que los nodos tengan acceso uniforme al almacenamiento compartido antes de desplegar servicios que dependan de él.

El playbook swarm-init.yml inicializa Docker Swarm. En primer lugar, ejecuta docker swarm init sobre el primer manager. Luego obtiene los tokens necesarios para unir managers adicionales y workers. Finalmente, une los nodos restantes y verifica el estado del clúster.

En conjunto, estos playbooks convierten un conjunto de instancias EC2 recién creadas en una plataforma de orquestación funcional, siendo una automatización que reduce la complejidad operativa y asegura que los nodos compartan una configuración homogénea.

## 5.5 Orquestación de servicios con Docker Swarm

### 5.5.1 Justificación

S.L.O.T.H. utiliza Docker Swarm como plataforma de orquestación de contenedores. Esta decisión responde a la necesidad de contar con despliegue distribuido, manejo de réplicas, redes overlay, políticas de reinicio, actualizaciones graduales y una operación “relativamente simple” para el alcance del MVP.

Docker Swarm permite expresar la plataforma mediante archivos docker-stack.yml, donde se define el estado deseado de los servicios, redes, volúmenes, variables de entorno, réplicas y políticas de despliegue. Esto facilita mantener una descripción clara del entorno sin introducir una complejidad operativa excesiva.

Para el estado actual del proyecto, Swarm ofrece un equilibrio adecuado entre capacidades distribuidas y simplicidad. Permite ejecutar múltiples servicios, distribuirlos entre nodos, aplicar restricciones de ubicación y realizar rolling updates sin requerir una plataforma de orquestación más compleja.

### 5.5.2 Definición de stacks por ambiente

Los stacks de Swarm se organizan por ambiente en los siguientes archivos:

- iac/swarm/development/docker-stack.yml
- iac/swarm/staging/docker-stack.yml
- iac/swarm/production/docker-stack.yml

Cada archivo representa el estado deseado de un ambiente específico. Aunque los ambientes comparten la misma estructura general, pueden variar en número de réplicas, variables de entorno, credenciales, dominios públicos, rutas de certificados, límites de recursos, políticas de exposición y parámetros de observabilidad.

Esta separación permite que development, staging y production evolucionen de manera controlada. El ambiente de desarrollo puede priorizar rapidez de iteración, staging puede utilizarse para validación previa y production debe aplicar configuraciones más estrictas de disponibilidad, seguridad y monitoreo.

### 5.5.3 Servicios desplegados

#### *5.5.3.1 Entrada y enrutamiento*

El servicio principal de entrada es Traefik. Este componente actúa como reverse proxy, terminador TLS y punto de acceso externo a la plataforma. Expone los puertos 80 y 443, observa el estado del clúster Swarm y construye rutas dinámicas a partir de labels asociadas a los servicios. Esto permite desacoplar la exposición pública de los servicios internos; en lugar de publicar directamente cada microservicio, el tráfico externo se concentra en un punto de entrada gestionado de forma minuciosa, desde donde se enruta hacia la Web App, el Core API u otros servicios autorizados.

#### *5.5.3.2 Observabilidad*

La capa de observabilidad está compuesta por Prometheus, Grafana, Loki, Tempo, Alloy, cAdvisor, Node Exporter y el servicio scaler.

Prometheus recolecta métricas de servicios, contenedores y nodos. Grafana permite visualizar métricas, logs y trazas mediante dashboards. Loki centraliza logs, Tempo almacena trazas distribuidas y Alloy funciona como colector y reenviador de telemetría. cAdvisor expone métricas de contenedores, mientras que Node Exporter expone métricas del host. El servicio scaler consulta métricas y puede aplicar acciones de escalamiento sobre Swarm.

Esta capa permite observar el comportamiento de la plataforma, detectar fallos, analizar consumo de recursos, estudiar cuellos de botella y evaluar la salud del sistema.

#### *5.5.3.3 Servicios de parsing*

Los servicios de parsing incluyen formula-parser, ddl-generator, sql-builder y excel-reader. Estos componentes se despliegan como servicios independientes conectados mediante redes overlay internas.

Su despliegue separado permite que cada etapa del pipeline de parsing pueda escalar, reiniciarse o evolucionar de forma independiente. Por ejemplo, si el análisis de fórmulas se convierte en un cuello de botella, el servicio correspondiente puede aumentar sus réplicas sin modificar necesariamente el Core API o la Web App.

#### *5.5.3.4 Almacenamiento y middleware*

El conjunto de almacenamiento y middleware incluye RabbitMQ, PostgreSQL, MongoDB, Redis y Database Service.

RabbitMQ funciona como centro asincrónico de la plataforma, PostgreSQL mantiene persistencia relacional y puede actuar como base destino por defecto, MongoDB almacena esquemas JSON, versiones y estructuras documentales, Redis funciona como caché operacional de baja latencia. Database Service actúa como proxy gRPC para unificar acceso a Redis y MongoDB.

Estos servicios son fundamentales para el funcionamiento del sistema, por lo que deben contar con volúmenes, redes y políticas de reinicio adecuadas.

#### *5.5.3.5 Aplicación y procesamiento*

Los servicios de aplicación y procesamiento son: prestart, api, typechecking y web.

El servicio prestart se ejecuta como replicated-job para preparar el estado inicial del sistema. Puede encargarse de migraciones, bootstrap o validaciones previas antes de iniciar el API principal.

El servicio api expone la lógica de negocio y coordina operaciones administrativas y asincrónicas. El servicio typechecking ejecuta workers de validación, inserción y resultados. El servicio web expone la interfaz de usuario para interacción con la plataforma.

### 5.5.4 Políticas operativas de Swarm

Los servicios definidos en Swarm incorporan políticas operativas que regulan su comportamiento durante despliegues, fallos y actualizaciones.

Las réplicas configurables permiten ajustar la capacidad de ejecución de cada servicio según el ambiente y la carga esperada. Las políticas de update_config habilitan despliegues graduales, evitando reemplazar todas las réplicas simultáneamente. Las políticas de rollback_config permiten volver a una versión anterior cuando una actualización falla o presenta comportamiento inesperado.

Las restart policies aseguran que los servicios puedan reiniciarse automáticamente ante fallos. Las placement constraints permiten controlar en qué nodos se ejecutan determinados servicios, por ejemplo, restringiendo servicios críticos a managers o distribuyendo workers en nodos específicos. Las reservas y límites de recursos ayudan a evitar que un servicio consuma recursos de forma descontrolada y afecte a los demás componentes.

Estas políticas hacen que la infraestructura no solo defina qué servicios existen, sino cómo deben comportarse en operación normal, durante despliegues y ante fallos.

## 5.6 Segmentación de red

La plataforma define varias redes overlay especializadas, entre ellas monitoring, parsers, redis, postgres, mongo, rabbit, grpc y traefik_proxy.

Esta segmentación responde a una decisión de seguridad, organización y mantenibilidad. En lugar de conectar todos los servicios a una red única, cada dominio funcional cuenta con una red propia. Esto reduce la superficie de comunicación, evita dependencias innecesarias y facilita razonar sobre qué componentes pueden comunicarse entre sí.

La red traefik_proxy concentra los servicios que requieren exposición hacia el exterior mediante Traefik. La red parsers conecta los componentes internos del subsistema de parsing. Las redes postgres, mongo, redis y rabbit aíslan los servicios de persistencia, caché y mensajería. La red grpc permite comunicaciones internas de baja latencia entre servicios que utilizan contratos tipados. La red monitoring da soporte a la recolección de métricas, logs y trazas.

Esta organización mejora el control operativo y permite aplicar políticas más precisas de comunicación. Además, refleja la topología funcional de la plataforma, haciendo que la infraestructura sea más comprensible y mantenible.

## 5.7 Seguridad operativa

### 5.7.1 Seguridad de red

La seguridad de red se aplica principalmente mediante Security Groups de AWS y segmentación interna de redes overlay. En el plano de AWS, solo se habilitan los puertos necesarios para la operación del clúster y la exposición controlada del sistema.

Los puertos HTTP y HTTPS permiten el acceso externo a la plataforma mediante Traefik. El puerto SSH se restringe a CIDRs autorizados para tareas administrativas. Los puertos de Docker Swarm, como 2377/tcp, 7946/tcp/udp y 4789/udp, habilitan la comunicación interna requerida para el plano de control, gossip y redes overlay. Cuando EFS está habilitado, el puerto 2049/tcp permite el montaje NFS entre nodos autorizados.

Esta configuración reduce la superficie de exposición y evita publicar servicios internos directamente hacia Internet.

### 5.7.2 Seguridad de acceso y certificados

La infraestructura de producción contempla el uso de certificados TLS para cifrar la comunicación externa. La ruta *iac/swarm/production/certs/* define el esquema esperado para certificados como domain.key y fullchain.crt.

Durante el despliegue, GitHub Actions puede reconstruir temporalmente estos archivos a partir de secretos protegidos, evitando que las llaves privadas o certificados se almacenen directamente en el repositorio. Posteriormente, Traefik utiliza dichos certificados para operar con terminación TLS en producción.

Este enfoque permite integrar seguridad de transporte sin comprometer material sensible en el control de versiones.

### 5.7.3 Seguridad del plano de gestión

Las instancias EC2 cuentan con un IAM Instance Profile con permisos asociados a AmazonSSMManagedInstanceCore. Esto habilitas capacidades de administración gestionada mediante AWS Systems Manager y reduce la dependencia exclusiva de sesiones SSH manuales.

El uso de IAM permite controlar qué acciones pueden ejecutar las instancias y bajo qué condiciones. Además, facilita prácticas de operación más seguras, al separar permisos de infraestructura de credenciales embebidas en los servicios de aplicación.

### 5.7.4 Gestión de secretos y variables de entorno

Los secretos de despliegue, certificados, credenciales de servicios y variables sensibles deben gestionarse fuera del código fuente. En el pipeline, GitHub Actions actúa como punto de inyección controlada de secretos hacia los procesos de construcción y despliegue.

En los stacks de Swarm, las variables de entorno permiten parametrizar servicios por ambiente. Esta separación evita modificar el código de aplicación para cambiar credenciales, rutas, dominios o configuraciones operativas.

## 5.8 Persistencia y datos operacionales

Aunque la arquitectura lógica define el rol funcional de PostgreSQL, MongoDB, Redis y RabbitMQ, desde infraestructura estos componentes deben analizarse como servicios operativos que requieren volúmenes, redes, políticas de reinicio, credenciales y monitoreo.

### 5.8.1 PostgreSQL

PostgreSQL se utiliza para persistencia relacional del Core API y, cuando aplica, como base de datos destino por defecto. Contiene entidades transaccionales como usuarios, proyectos, relaciones entre usuarios, proyectos y tareas de carga.

Desde la infraestructura, PostgreSQL debe desplegarse con volumen persistente, credenciales configuradas por ambiente y conectividad restringida a la red interna correspondiente. Su disponibilidad es crítica, ya que almacena hitos formales del proceso, incluyendo estados finales de tareas.

### 5.8.2 MongoDB

MongoDB soporta la persistencia documental de esquemas JSON, versiones de esquemas y respaldo de estados operacionales. Su flexibilidad lo hace adecuado para almacenar JSON Schemas y estructuras cuyo formato puede variar entre proyectos o tablas.

A nivel de despliegue, MongoDB debe contar con volumen persistente y autenticación configurada. La pérdida de esta información afectaría la capacidad del sistema para validar archivos bajo reglas previamente definidas y auditar cambios en reglas de negocio.

### 5.8.3 Redis

Redis se utiliza como caché operacional y almacén de estados temporales con TTL. Su objetivo principal es proporcionar acceso rápido a información de ejecución, no funcionar como fuente de persistencia duradera.

Por esta razón, la infraestructura debe tratar Redis como un acelerador. Una falla en Redis no debería comprometer de manera definitiva la integridad de reglas, tareas o resultados críticos, siempre que estos estén respaldados en PostgreSQL o MongoDB.

### 5.8.4 RabbitMQ

RabbitMQ actúa como componente central del procesamiento asincrónico. Permite desacoplar la solicitud del usuario del trabajo pesado ejecutado por workers, distribuyendo tareas de validación, inserción y consolidación de resultados.

Desde la infraestructura, RabbitMQ debe considerarse un servicio crítico. Su indisponibilidad afecta directamente la capacidad del sistema para iniciar o continuar tareas asincrónicas. Por ello, debe contar con políticas de reinicio, monitoreo y persistencia de colas cuando el ambiente lo requiera.

## 5.9 Observabilidad y operación

La observabilidad es una capacidad operacional integrada en la infraestructura de S.L.O.T.H., ya que, debido a que el sistema ejecuta procesos distribuidos, asincrónicos y potencialmente prolongados, es

necesario contar con mecanismos que permitan entender el comportamiento de los servicios durante la operación.

### 5.9.1 Recolección de métricas

- Prometheus recolecta métricas expuestas por servicios de aplicación, contenedores y nodos. Los endpoints /metrics permiten que los microservicios publiquen información relevante sobre solicitudes, tiempos de ejecución, errores, tareas procesadas y estado interno.
- cAdvisor expone métricas relacionadas con contenedores, como consumo de CPU, memoria, red y disco. Node Exporter expone métricas del host, permitiendo analizar capacidad disponible, presión de recursos y comportamiento del sistema operativo.
- Estas métricas permiten identificar cuellos de botella, evaluar escalabilidad y detectar servicios que consumen recursos de forma anómala.

### 5.9.2 Logs y trazas

- Loki centraliza logs generados por servicios y contenedores. Esto permite consultar errores, advertencias y eventos operativos sin tener que conectarse individualmente a cada nodo del clúster.
- Tempo almacena trazas distribuidas, lo que permite reconstruir el recorrido de una solicitud o tarea a través de varios servicios. Esta capacidad es especialmente importante en la arquitectura, ya que, por ejemplo, una operación puede iniciar en la Web App, pasar por Core API, publicarse en RabbitMQ, ejecutarse en un worker, consultar Database Service y finalmente cerrarse mediante un callback.
- Alloy actúa como colector y reenviador de telemetría, integrando logs, métricas y trazas hacia los sistemas correspondientes.

### 5.9.3 Visualización operacional

La visualización operacional permite que el equipo de gestión de infraestructura identifique degradaciones antes de que se conviertan en fallos críticos. También facilita la evaluación de pruebas de carga y la toma de decisiones sobre escalamiento.

En este contexto, Grafana centraliza dashboards de métricas, logs y trazas. Desde esta herramienta se puede observar el estado de los servicios, la salud del clúster, el rendimiento de los workers, la ocupación de colas, el consumo de recursos y los errores de aplicación.

### 5.9.4 Autoscaling

El servicio scaler consulta métricas de Prometheus y aplica acciones de escalamiento sobre Docker Swarm a través del socket Docker. Su comportamiento puede configurarse mediante intervalos, umbrales y períodos de cooldown para evitar escalados excesivos o inestables.

Este mecanismo permite adaptar parcialmente la capacidad del sistema a la carga observada, especialmente en servicios de procesamiento que pueden requerir más réplicas cuando aumenta el número de tareas pendientes o el uso de recursos.

## 5.10 Pipeline de entrega continua

### 5.10.1 Integración continua

Los workflows de integración continua se ubican en *.github/workflows/ci-*-tests.yml.* Estos workflows ejecutan pruebas por servicio y posteriormente construyen imágenes Docker cuando las validaciones son exitosas.

El patrón general consiste en ejecutar comandos como moon run service:test para validar el comportamiento del componente, seguido de moon run service:docker-build para construir la imagen correspondiente.

Esta organización permite validar microservicios de manera independiente antes de integrarlos al flujo completo de despliegue. Además, reduce el riesgo de publicar imágenes defectuosas hacia los ambientes superiores.

### 5.10.2 Versionado y promoción entre ambientes

El pipeline distingue entre ambientes de development, staging y production. Cada ambiente tiene sus propios workflows, variables, secretos y stacks de Swarm.

Cuando los cambios se fusionan hacia ramas de promoción, el pipeline puede crear tags, construir imágenes, publicarlas en Docker Hub, actualizar referencias en los archivos docker-stack.yml y disparar el despliegue hacia el clúster correspondiente. Esta estrategia permite controlar el paso de cambios desde desarrollo hasta producción.

### 5.10.3 Despliegue remoto al clúster

El action reutilizable *.github/actions/deploy-cluster/action.yml* encapsula la lógica de despliegue hacia el clúster. Su flujo incluye configuración del agente SSH, carga de variables de ambiente, creación de un docker context remoto sobre SSH, ejecución del comando de despliegue correspondiente mediante Moon y limpieza de archivos temporales.

Este diseño evita que el despliegue dependa de acceso manual interactivo al clúster. En su lugar, el repositorio contiene el flujo necesario para construir, versionar y desplegar servicios de manera controlada.

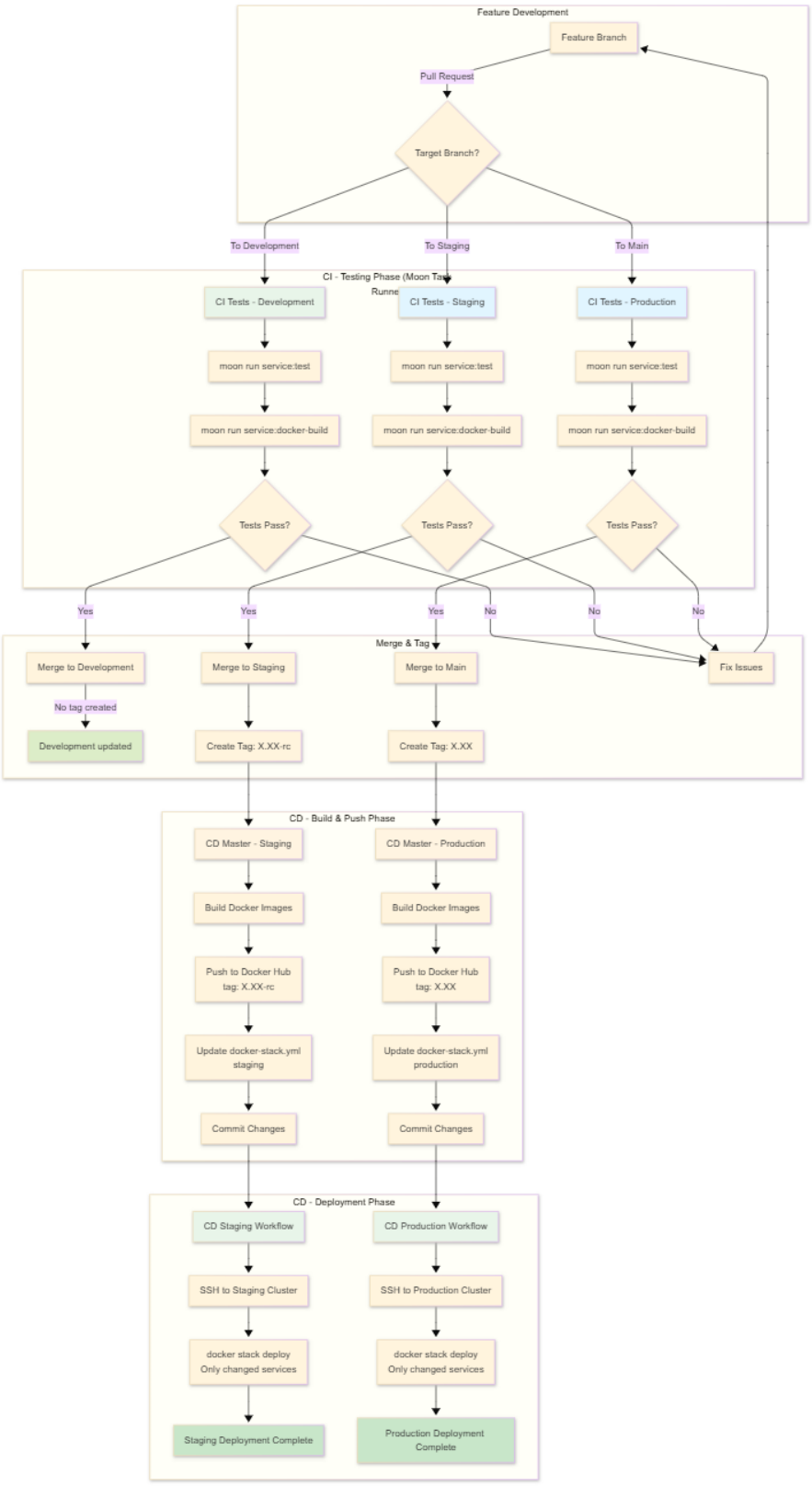



*Imagen 8: Flujo de CI/CD.*

## 5.11 Limitaciones y consideraciones actuales

La infraestructura descrita corresponde a una versión MVP del proyecto, por lo tanto, algunas decisiones priorizan rapidez de implementación, claridad operativa y facilidad de despliegue sobre sofisticación extrema.

Docker Swarm es el orquestador activo. Aunque resulta adecuado para la versión actual, una migración futura hacia Kubernetes u otra plataforma requeriría rediseñar parte del despliegue, especialmente los stacks, constraints, redes y políticas de actualización.

El almacenamiento compartido mediante EFS cubre necesidades operativas del proyecto, pero podría requerir evaluación adicional en escenarios de alta concurrencia, mayor volumen de datos o requisitos estrictos de disponibilidad regional.

La separación por múltiples archivos de ambiente y workflows otorga control fino, pero también incrementa el esfuerzo de mantenimiento. A medida que el sistema crezca, será necesario revisar mecanismos de reutilización, plantillas o generación automatizada para evitar divergencias entre ambientes.

Finalmente, los servicios de persistencia desplegados dentro del clúster deben evaluarse cuidadosamente antes de una operación productiva de alta criticidad. En escenarios empresariales exigentes, podría considerarse migrar PostgreSQL, MongoDB, Redis o RabbitMQ hacia servicios administrados o configuraciones de alta disponibilidad dedicadas.

# 6. Decisiones y criterios relevantes

Durante el desarrollo del proyecto, se tomaron múltiples decisiones tecnológicas, arquitectónicas y de infraestructura que responden a requerimientos, objetivos y condiciones varias del proyecto. Aunque puede que en esta sección no encuentre respuesta o descripción a todas sus preguntas, sí se justifican los criterios fundamentales (o los que fueron considerados fundamentales durante el desarrollo, en función del problema que S.L.O.T.H intenta abordar).

Dicho esto, la siguiente sección se compone de pares decisión/pregunta – respuesta/justificación para las áreas de arquitectura e infraestructura del proyecto.

## 6.1 Arquitectura

### 6.1.1 Adoptar infraestructura basada en microservicios

S.L.O.T.H. adopta una arquitectura basada en microservicios colaborativos, en la cual las responsabilidades principales se distribuyen entre la Web App, el Core API, el Typechecking Service, el Parsing Subsystem y el Database Service.

Esto se debe a que el sistema no resuelve un único flujo lineal, sino un conjunto de capacidades diferenciadas: autenticación, gestión de proyectos, administración de esquemas, validación de archivos, inserción en bases de datos, traducción de fórmulas, persistencia de reglas y consulta de estados. Concentrar todas estas responsabilidades en una aplicación monolítica incrementaría el acoplamiento, dificultaría el mantenimiento y limitaría la posibilidad de escalar los componentes que ejecutan procesamiento intensivo.

La separación por microservicios permite que cada bloque evolucione de acuerdo con sus propias necesidades. Por ejemplo, el subsistema de parsing puede incorporar nuevas funciones de Excel sin afectar la autenticación o la gestión de proyectos, mientras que los workers de validación pueden escalarse sin modificar la interfaz de usuario.

***¿Por qué no un monolito?***

Porque el sistema incluye tareas con perfiles muy distintos: operaciones administrativas de baja latencia, procesamiento asincrónico de archivos, análisis sintáctico de fórmulas y persistencia de estados distribuidos. Un monolito simplificaría el despliegue inicial, pero aumentaría el acoplamiento y dificultaría el escalamiento selectivo.

***¿Qué se gana con los microservicios?***

Se gana separación de responsabilidades, despliegue independiente, escalabilidad por componente, aislamiento de fallos y mayor claridad para evolucionar módulos especializados.

***¿Qué costos implica esta decisión?***

Introduce mayor complejidad operacional: comunicación entre servicios, observabilidad distribuida, manejo de fallos parciales y necesidad de contratos claros entre componentes. Adicionalmente, soportar microservicios también podría llegar a ocasionar un despliegue en nube económicamente más costoso, ya que gestionar múltiples máquinas virtuales o contenedores implica mayores requerimientos computacionales.

### 6.1.2 Separar operaciones sincrónicas y asincrónicas

El sistema separa las operaciones de control, administración y consulta de las operaciones pesadas de validación, inserción y generación de resultados. Las primeras se atienden mediante APIs sincrónicas; las segundas se delegan a colas de mensajes y workers.

Se ejecutó de esta manera ya que la carga y validación de archivos puede requerir tiempos variables según el número de filas, columnas, reglas aplicadas, fórmulas detectadas y disponibilidad de la base destino. Ejecutar estas operaciones dentro del ciclo de una solicitud HTTP generaría bloqueos, timeouts y una mala experiencia de usuario. Por ello, el Core API registra la tarea, devuelve un identificador y delega el procesamiento a RabbitMQ y a los workers correspondientes.

***¿Por qué no procesar el archivo directamente cuando el usuario lo carga?***

Porque el tiempo de procesamiento puede ser, aunque predecible, excesivamente extenso. La respuesta inmediata con un task_id permite que el usuario continúe trabajando y consulte el estado posteriormente.

***¿Qué garantías ofrece el procesamiento asincrónico?***

Permite desacoplar recepción y ejecución, distribuir carga entre workers, aplicar reintentos, registrar estados intermedios y escalar la capacidad de procesamiento.

***¿Introduce algún riesgo?***

Introduce consistencia eventual. El sistema debe manejar estados como pendiente, en procesamiento, completado o fallido, y debe ofrecer mecanismos de consulta para que el usuario conozca el avance de la tarea, pero esto es algo que el procesamiento sincrónico también ocasiona en cierta medida, tiene mas que ver con condiciones inherentes al tamaño de las tareas.

### 6.1.3 Utilizar RabbitMQ como centro para el procesamiento asincrónico

S.L.O.T.H. utiliza RabbitMQ como mecanismo de mensajería para coordinar las etapas de validación, inserción y consolidación de resultados.

Como tecnología, RabbitMQ permite desacoplar temporalmente los servicios que reciben solicitudes de los servicios que ejecutan trabajo pesado. Además, ofrece un modelo adecuado para colas de tareas, confirmación de mensajes, reintentos, distribución de carga y separación de flujos por tipo de operación.

En el contexto de la arquitectura, RabbitMQ permite que los workers consuman tareas según disponibilidad, evitando que el Core API tenga que ejecutar directamente procesos intensivos. Esta decisión está alineada con los requerimientos de rendimiento, escalabilidad y resiliencia definidos previamente en el documento.

***¿Por qué una cola de mensajes?***

Porque los procesos ETL pueden ser largos y deben tolerar fallos, reintentos y variación en la carga. También se hace casi obligatorio por buenas prácticas al tener procesamiento intensivo gestionado de manera asincrónica.

***¿Por qué RabbitMQ y no llamadas HTTP entre servicios?***

Las llamadas HTTP directas acoplan temporalmente a los servicios: si un worker o servicio destino no está disponible, la operación falla de inmediato. RabbitMQ permite almacenar la tarea y procesarla cuando exista capacidad.

***¿Qué consecuencia/s tiene esta decisión?***
El sistema gana resiliencia y elasticidad, pero requiere monitorear colas, mensajes fallidos y tiempos de procesamiento. Se gana en disponibilidad y resiliencia, pero se aumenta el monitoreo y mantenimiento.

### 6.1.4 Utilizar gRPC para comunicación interna entre servicios core

Las comunicaciones internas de alto rendimiento y contratos estrictos se implementan mediante gRPC y Protocol Buffers.

La arquitectura comunicación frecuente entre servicios internos, especialmente entre el Core API, los workers, el Database Service y los servicios del subsistema de parsing. Estas interacciones manejan estructuras que deben ser consistentes, como estados de tarea, esquemas, AST, expresiones SQL y respuestas de procesamiento. gRPC permite definir contratos tipados, reducir ambigüedades y disminuir la sobrecarga de serialización frente a alternativas menos estrictas.

***¿Por qué no usar REST para todo?***
REST es adecuado para la frontera externa y la interoperabilidad con la Web App, pero no siempre es la opción más eficiente, segura ni auditable para comunicación interna frecuente entre servicios.

***¿Qué aporta gRPC?***
Contratos tipados, eficiencia en serialización, compatibilidad con comunicación entre lenguajes y una interfaz más clara para operaciones internas.

***¿Dónde se usa principalmente?***
En la comunicación entre servicios del parser, entre el Core API y el Database Service, y entre workers y servicios internos que requieren baja latencia o contratos estrictos.

### 6.1.5 Almacenar reglas de validación en Json schemas

Las reglas de validación del sistema se representan mediante JSON Schemas, esto permite definir de forma estructurada los tipos, campos obligatorios, restricciones de longitud, rangos, patrones y condiciones esperadas sobre los datos. Su uso permite que las reglas sean interpretables por el sistema, almacenables en un repositorio documental y gestionables desde una interfaz no-code.

Es una de las decisiones fundamentales de S.L.O.T.H y se basa en uno de sus objetivos primarios: desacoplar las reglas de validación del código fuente para que puedan ser administradas, versionadas y auditadas sin necesidad de modificar directamente los scripts ETL.

***¿Por qué no mantener las reglas de validación en el código?***
Porque las reglas de negocio cambian con frecuencia y muchas veces son conocidas por usuarios funcionales. Si están embebidas en código, cualquier cambio exige intervención técnica, pruebas y redeploy. Esta sección se explora más a profundidad en los artículos anexos a este documento.

***¿Qué se gana con JSON Schema?***
Se obtiene una representación estándar, estructurada, validable y almacenable de las reglas.

***¿Cuál es la limitación?***

Algunas reglas complejas pueden no expresarse directamente con JSON Schema básico y podrían requerir extensiones, validadores personalizados o reglas compuestas.

### 6.1.6 Almacenamiento y versionamiento de reglas en MongoDB

Las reglas de validación y sus versiones se almacenan en MongoDB mediante estructuras documentales como active_schema y schemas_releases. Esto se debe a que los JSON Schemas son documentos con estructura flexible: pueden variar en cantidad de campos, tipos de restricciones y complejidad según cada tabla o proyecto.

MongoDB se ajusta bien a este modelo porque permite almacenar documentos sin forzar una estructura relacional rígida, además, el versionamiento de reglas requiere conservar versiones históricas, recuperar esquemas activos y permitir reversión. Un modelo documental facilita mantener cada versión como un documento completo, preservando trazabilidad y simplificando la consulta de esquemas. En el documento actual, MongoDB se define como soporte para esquemas activos, historial de versiones y estructuras flexibles relacionadas con reglas de validación.

***¿Por qué MongoDB y no PostgreSQL, que ya está en la arquitectura, para almacenar las reglas?***

PostgreSQL podría almacenar JSON, pero MongoDB ofrece un modelo documental más natural para esquemas variables y versiones completas de documentos.

### 6.1.7 Uso de Redis como caché operacional

Redis ofrece baja latencia y es adecuado para almacenar estados efímeros, resultados parciales, llaves temporales y datos de coordinación. Sin embargo, su naturaleza lo hace menos apropiado como única fuente de información crítica. Por ello, la arquitectura define que la información relevante debe respaldarse en repositorios duraderos como PostgreSQL o MongoDB.

Esta decisión ya se refleja en la sección de arquitectura, donde Redis se plantea como acelerador y no como fuente de datos determinante.

***¿Por qué no depender exclusivamente de Redis?***

Porque una falla o limpieza de caché no debe comprometer la trazabilidad ni la integridad del procesamiento ETL.

### 6.1.8 Diseño de workers sin estado persistente propio

Los workers se diseñan como unidades de ejecución que no conservan estado persistente local. El estado del procesamiento se almacena externamente. Esto se debe a que, en una arquitectura distribuida, los workers pueden reiniciarse, escalarse, fallar o ser reemplazados. Si el estado dependiera del worker local, una falla podría provocar pérdida de información o imposibilidad de recuperar tareas. Al externalizar el estado, el sistema puede reintentar procesos, mover tareas entre workers y escalar instancias sin perder trazabilidad.

***¿Qué pasa si un worker falla?***

La tarea puede reintentarse o marcarse como fallida según la política definida, porque el estado no depende exclusivamente del proceso caído.

***¿Qué se obtiene con esta metodología?***

Escalabilidad horizontal, reinicios controlados, recuperación ante fallos y menor acoplamiento entre ejecución y persistencia.

### 6.1.9 Incorporar idempotencia como principio base del pipeline

El sistema genera claves de idempotencia a partir del archivo y su contexto de ejecución para evitar duplicidad de tareas o cargas, pues en flujos asincrónicos pueden ocurrir reintentos, solicitudes duplicadas, fallos de red o reprocesamientos involuntarios.

Sin idempotencia, el mismo archivo podría insertarse más de una vez en la base destino, comprometiendo la calidad del data warehouse. Por esta razón, S.L.O.T.H. calcula hashes y registra tareas para detectar cargas repetidas o estados activos.

***¿Qué problema resuelve la idempotencia y por qué es relevante en el flujo ETL?***

En resumen, la idempotencia evita que una misma operación ejecutada varias veces produzca efectos repetidos, lo que es fundamental ya que una duplicación puede contaminar el repositorio analítico y afectar reportes o decisiones posteriores.

### 6.1.10 Priorizar disponibilidad y tolerancia a particiones en procesos asincrónicos

En los flujos asincrónicos de validación, inserción y consolidación de resultados, S.L.O.T.H. prioriza la disponibilidad del sistema y la tolerancia a fallos parciales por encima de una consistencia estricta e inmediata entre todos los componentes distribuidos.

De acuerdo con el teorema CAP, en un sistema distribuido no es posible garantizar simultáneamente consistencia fuerte, disponibilidad y tolerancia a particiones ante fallos de comunicación. En nuestro caso, los procesos ETL pueden atravesar múltiples componentes: Core API, RabbitMQ, workers, Database Service, Redis, MongoDB, PostgreSQL y servicios de parsing. Por esta razón, imponer una transacción global estricta entre todos los servicios aumentaría el acoplamiento, reduciría la disponibilidad y dificultaría la recuperación ante fallos parciales.

La arquitectura opta por una estrategia de consistencia eventual controlada para los estados intermedios del pipeline. Esto significa que, durante el procesamiento de una tarea, diferentes componentes pueden observar estados transitorios distintos, como PENDING, PROCESSING o resultados parciales. Sin embargo, los hitos finales del proceso, como COMPLETED, FAILED o PARTIALLY_COMPLETED, deben consolidarse formalmente mediante el Core API y persistirse en PostgreSQL como fuente transaccional de cierre.

Esta decisión permite que el sistema continúe operando aunque existan fallos temporales en workers, retrasos en colas, indisponibilidad momentánea de servicios auxiliares o inconsistencias temporales en caché. En lugar de detener todo el flujo ante una falla parcial, S.L.O.T.H. utiliza reintentos, colas, estados externos e idempotencia para alcanzar un resultado final consistente.

***¿Por qué no usar consistencia fuerte en todo el sistema?***

Porque exigir consistencia fuerte entre todos los servicios implicaría coordinar transacciones distribuidas entre componentes heterogéneos, como colas, bases de datos, caché y workers. Esto aumentaría la complejidad y reduciría la disponibilidad del sistema.

***¿Dónde se acepta consistencia eventual?***

En estados intermedios del procesamiento asincrónico, como progreso de tareas, resultados parciales, mensajes en cola, caché operacional y coordinación entre workers.

***¿Dónde sí se requiere consistencia fuerte?***

En los hitos formales del proceso, especialmente al registrar tareas, evitar duplicados mediante idempotencia, consolidar estados finales y persistir resultados definitivos en PostgreSQL.

***¿Qué riesgo introduce?***

El usuario o los servicios internos pueden observar estados temporales desactualizados durante una ventana corta. Este riesgo se mitiga mediante task_id, consulta de estado, trazabilidad, reintentos e idempotencia.

## 6.2 Infraestructura

### 6.2.1 Uso de infraestructura como código

Actualmente la infraestructura se define mediante archivos versionados de IaC, utilizando Terraform para aprovisionamiento, Ansible para configuración, Docker Swarm para orquestación y GitHub Actions para entrega continua.

Teniendo en cuenta que S.L.O.T.H. requiere ambientes reproducibles y consistentes, una configuración manual de servidores, redes, permisos y despliegues aumentaría el riesgo de diferencias entre ambientes y dificultaría la auditoría de cambios, lo que la hace poco viable y técnicamente ineficiente para el caso. Por otro lado, la infraestructura como código permite revisar, versionar y reconstruir el entorno de forma controlada e idéntica cada vez.

### 6.2.2 Aprovisionamiento sobre AWS

En este momento, la infraestructura base se despliega sobre AWS, utilizando recursos como VPC, subredes, Internet Gateway, Security Groups, EC2, IAM Instance Profiles y EFS.

AWS proporciona los recursos necesarios para desplegar una plataforma distribuida con control de red, cómputo escalable, almacenamiento compartido, gestión de permisos y administración remota. Además, permite separar ambientes y parametrizar infraestructura mediante Terraform.

***¿Por qué usar una nube pública?***

Porque facilita aprovisionar recursos distribuidos, escalar capacidad y automatizar entornos sin depender de infraestructura física local.

***¿Por qué AWS?***

Realmente, por nada en particular. AWS ofrece un ecosistema consolidado e integrado gestionable a través de línea de comandos que, además, tiene integraciones nativas con Terraform, pero estas condiciones

también son ciertas para otros grandes prestadores de nube como Azure u Oracle, e incluso con empresas de menor envergadura como Digital Ocean o Vultr.

El uso de AWS no es obligatorio, y con cambios en los archivos de Terraform (que, valga decirlo, serían bastante extensivos) podría migrarse a otra plataforma sin problemas.

### 6.2.3 Separar en mútiples nodos manager y workers en Docker Swarm

El clúster distingue entre nodos manager y nodos worker, donde los managers mantienen el estado del clúster y coordinan operaciones administrativas, mientras que los workers ejecutan cargas de aplicación y procesamiento. Esta separación evita mezclar responsabilidades de gobierno del clúster con ejecución de tareas intensivas.

El plano de control del clúster requiere estabilidad, mientras que los workers deben poder escalar para absorber carga. Esta separación permite ubicar servicios críticos en managers, distribuir procesamiento en workers y aplicar restricciones de despliegue según el tipo de servicio.

### 6.2.4 Usar Docker Swarm como orquestador

Docker Swarm ofrece capacidades suficientes para el estado actual del proyecto: despliegue distribuido, redes overlay, manejo de réplicas, rolling updates, restart policies y restricciones de ubicación. Para un MVP funcional, Swarm reduce la complejidad operativa frente a alternativas más robustas, pero también más exigentes en mantenimiento y computación.

***¿Por qué no utilizar Kubernetes?***

Kubernetes ofrece mayores capacidades, pero también introduce más complejidad en configuración, operación, monitoreo y requiere mayor capacidad computacional. Para el alcance actual, Swarm proporciona un balance adecuado entre distribución y simplicidad.

### 6.2.5 Usar Traefik como punto de entrada y reverse proxy

Traefik permite concentrar la exposición externa en un solo componente, enrutar tráfico hacia servicios internos y administrar certificados TLS. Además, se integra con Docker Swarm y puede construir rutas a partir de labels de servicios.

***¿Por qué no exponer cada servicio directamente?***

Porque aumentaría la superficie de ataque, dificultaría la gestión de certificados y haría más compleja la administración de rutas, además, en términos generales, no es realmente necesario, muchos servicios son internos y no deberían estar expuestos a la red.

### 6.2.6 Segmentar redes overlay por dominio funcional

La segmentación de red reduce la comunicación innecesaria entre servicios y limita la superficie de exposición interna. No todos los servicios necesitan comunicarse entre sí; por tanto, cada componente debe conectarse solo a las redes que requiere para cumplir su función.

### 6.2.7 Utilizar EFS como almacenamiento compartido

En un clúster distribuido, los contenedores pueden ejecutarse en distintos nodos. Algunos recursos, como certificados TLS, artefactos compartidos o configuraciones persistentes, necesitan estar disponibles

independientemente del nodo donde se ejecute el servicio. EFS permite compartir estos recursos sin depender del disco local.

***¿Por qué no utilizar un solo disco local?***

Porque si un servicio se reubica en otro nodo, podría perder acceso a los archivos necesarios para su operatividad.

### 6.2.8 Observabilidad extensiva en la infraestructura base

Como puede observarse en las secciones *5.5.3.2 y 5.9.3,* se utilizan múltiples tecnologías para observabilidad del sistema, y aunque a primera vista pueden parecer demasiadas, cada una de ellas es fundamental para el seguimiento de capas distintas de la arquitectura.

En un sistema distribuido y asincrónico como S.L.O.T.H, una operación puede atravesar Web App, Core API, RabbitMQ, workers, Database Service, servicios de parsing y bases de datos. Sin observabilidad profunda, diagnosticar fallos o cuellos de botella sería complejo. Por eso, la telemetría se integra desde la infraestructura base y no como una extensión posterior.

## 6.3 Criterios transversales de evaluación

Las decisiones anteriores no deben evaluarse de forma aislada. En conjunto, responden a criterios transversales que orientan el diseño del sistema:

- **Escalabilidad**: El sistema debe permitir aumentar capacidad de procesamiento mediante workers, réplicas y nodos adicionales, especialmente en validación, parsing e inserción.
- **Trazabilidad:** Cada carga debe poder rastrearse desde la solicitud inicial hasta su cierre, incluyendo reglas aplicadas, estados intermedios, errores y resultados.
- **Gobernanza:** Las reglas de negocio deben estar centralizadas, versionadas y desacopladas del código fuente, permitiendo auditoría y reversión.
- **Resiliencia:** El sistema debe tolerar fallos parciales mediante colas, reintentos, estado externo, políticas de reinicio y persistencia de resultados.
- **Mantenibilidad:** Los servicios deben tener responsabilidades claras, contratos explícitos y capacidad de evolucionar de forma independiente.
- **Seguridad operativa:** El sistema debe proteger credenciales, restringir acceso por roles, limitar exposición de red y evitar almacenamiento de secretos en texto plano.
- **Observabilidad:** La plataforma debe generar métricas, logs y trazas que permitan diagnosticar fallos y evaluar rendimiento.

# 7. Resultados

La evaluación de S.L.O.T.H. permite analizar el sistema desde cuatro perspectivas complementarias: los resultados funcionales obtenidos por la plataforma, los resultados de rendimiento, los resultados arquitectónicos derivados de la separación por microservicios y los resultados operativos asociados a infraestructura, despliegue y observabilidad.

Estos, en conjunto, evidencian que la solución propuesta no solo implementa un flujo ETL para hojas de cálculo, sino que también introduce mecanismos de gobernanza, trazabilidad, escalabilidad y migración de lógica de negocio hacia el data warehouse que pueden ejecutarse incluso sobre distintas fuentes de datos.

## 7.1 Resultados funcionales

Desde el punto de vista funcional, S.L.O.T.H. logra materializar una plataforma capaz de gestionar el ciclo completo de procesamiento de archivos tabulares dentro de un flujo ETL controlado. El sistema permite registrar usuarios, iniciar sesión, administrar perfiles, gestionar proyectos, configurar credenciales de conexión a bases de datos destino, asociar miembros a proyectos y controlar permisos mediante roles y atributos.

Adicionalmente, la plataforma permite crear, actualizar, consultar y revertir esquemas de validación definidos mediante JSON Schema. Esta capacidad constituye uno de los resultados funcionales más relevantes del proyecto, ya que permite que las reglas de validación dejen de estar embebidas directamente en scripts ETL y pasen a ser administradas como artefactos independientes, versionables y auditables. Los artículos base del proyecto destacan precisamente esta necesidad: en entornos ETL tradicionales, las reglas suelen estar dispersas, sin versionamiento o incluidas dentro del código, lo que dificulta su mantenimiento y aumenta la dependencia de perfiles técnicos.

El sistema también permite ejecutar validación asincrónica de archivos .xlsx, .xls y .csv, identificando registros válidos e inválidos antes de la inserción en la base de datos destino. Como resultado, S.L.O.T.H. evita que información inconsistente sea incorporada al repositorio analítico y genera reportes que permiten al usuario conocer los errores detectados durante el procesamiento.

Otro resultado funcional importante es la capacidad de generar SQL a partir de archivos Excel con fórmulas. El subsistema de parsing permite extraer fórmulas, construir representaciones intermedias como árboles de sintaxis abstracta y traducir dichas expresiones a sentencias SQL compatibles con PostgreSQL. Esta funcionalidad traslada parte de la lógica computacional desde el archivo cliente hacia el motor de base de datos, reduciendo la dependencia de hojas de cálculo como contenedores de lógica de negocio.

## 7.2 Resultados de validación de la interfaz no-code

Uno de los objetivos centrales de S.L.O.T.H. es reducir la dependencia de equipos técnicos para la creación y modificación de reglas de validación. Para esto se implementó una interfaz gráfica orientada a usuarios no técnicos, desde la cual es posible crear entidades de datos, definir columnas, seleccionar tipos, configurar restricciones y visualizar el JSON Schema resultante.

En los trabajos previos se evaluó esta interfaz mediante un estudio piloto con 20 participantes de distintos perfiles: analistas de datos, profesionales de marketing, estudiantes de ciencias de la computación y estudiantes de finanzas. Los participantes debían crear reglas de validación para un reporte de ventas utilizando la interfaz gráfica. El resultado global fue una tasa de éxito del 85 %, con un tiempo promedio de finalización de 6 minutos y 23 segundos, y un intento más rápido de 1 minuto y 36 segundos.

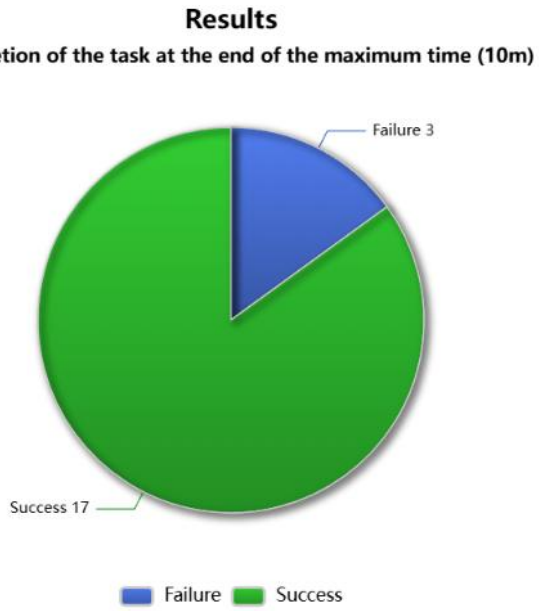



*Imagen 9: pruebas de interfaz con usuarios.*

Estos resultados muestran que la interfaz cumple adecuadamente su propósito para tareas comunes de definición de reglas. Las validaciones numéricas y booleanas recibieron una valoración positiva, mientras que las validaciones basadas en patrones de texto obtuvieron menor calificación debido a la complejidad propia de las expresiones regulares. Este hallazgo es relevante para el proyecto porque identifica una oportunidad clara de mejora: la construcción de un mecanismo visual para patrones, como un *Pattern Builder*, que permita abstraer la complejidad de las expresiones regulares y acercar aún más la herramienta a usuarios no técnicos.

## 7.3 Resultados de rendimiento y escalabilidad

La evaluación de rendimiento se realizó sobre archivos sintéticos en formato .xlsx, variando tanto el volumen de datos como la dimensionalidad. En los experimentos se midieron tiempos de ejecución bajo procesamiento lineal y procesamiento paralelo, considerando etapas como validación, lectura de metadatos, comunicación entre servicios, creación de tablas e inserción de datos.

En el primer escenario, se mantuvo fijo el número de filas en 1.000 y se varió el número de columnas entre 100 y 1.000. En este caso, la ejecución lineal pasó de 6,21 segundos para 100 columnas a 87,67 segundos para 1.000 columnas. Bajo ejecución paralela con 16 hilos, el tiempo pasó de 3,01 segundos a 57,37 segundos en el mismo rango de columnas.

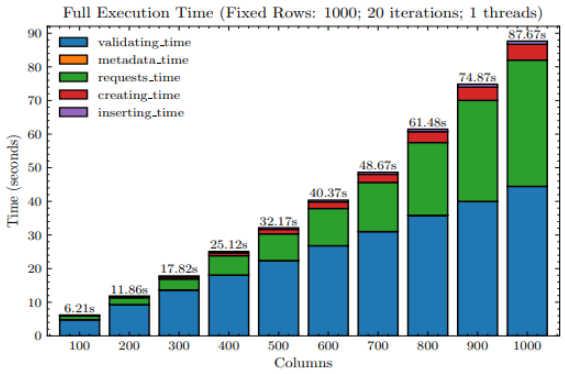



*Imagen 10: Tiempos de ejecución promedio para incrementos de dimensionalidad de datos en ejecución lineal.*

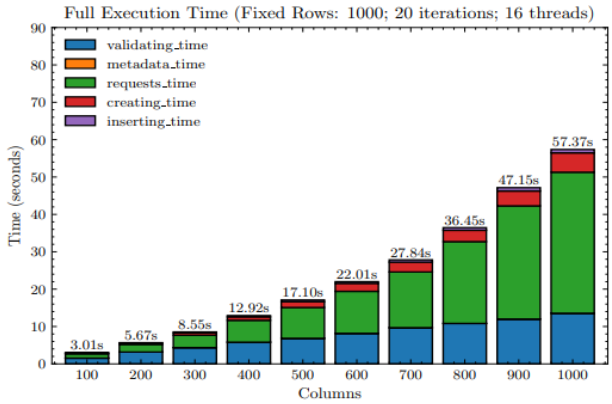



*Imagen 11: Tiempos de ejecución promedio para incrementos de dimensionalidad de datos en ejecución paralela a 16 hilos.*

En el segundo escenario, se mantuvo fijo el número de columnas en 100 y se varió el número de filas entre 1.000 y 10.000. La ejecución lineal pasó de 6,26 segundos para 1.000 filas a 53,89 segundos para 10.000 filas. Con procesamiento paralelo en 16 hilos, el tiempo se redujo de 3,27 segundos a 22,00 segundos para el mismo rango de datos.

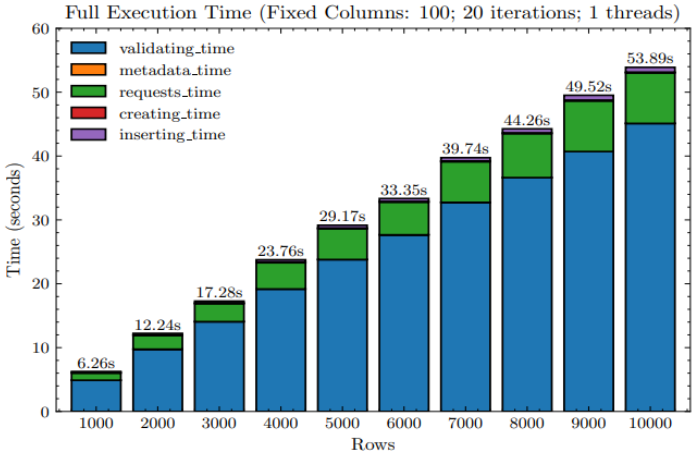



*Imagen 12: Tiempos de ejecución promedio para incrementos de volumen de datos en ejecución lineal.*

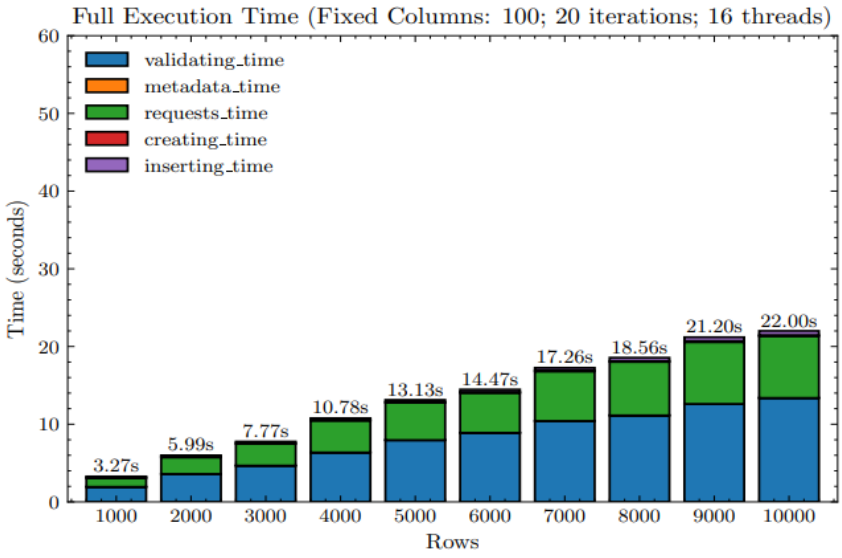



*Imagen 13: Tiempos de ejecución promedio para incrementos de volumen de datos en ejecución paralelizada a 16 hilos.*

Estos resultados evidencian que el sistema presenta un crecimiento predecible frente al aumento de filas y columnas. Aunque tanto la dimensionalidad como el volumen impactan el rendimiento, los experimentos muestran que la arquitectura escala de forma controlada y sin presentar cuellos de botella exponenciales en los escenarios evaluados.

La paralelización tuvo un impacto significativo en el rendimiento general. Para archivos con mayor dimensionalidad, la ejecución paralela mejoró el tiempo total en un promedio aproximado del 45 %. Para archivos con mayor volumen, la mejora alcanzó aproximadamente el 55 %, con mayor impacto a medida que aumentaba el número de filas. Además, la fase de validación presentó una reducción cercana al 70 %, lo que confirma la efectividad de dividir y consumir tareas mediante una cola de procesamiento y múltiples workers.

## 7.4 Resultados del sistema de parsing y migración de lógica

El subsistema de parsing constituye una de las contribuciones técnicas más relevantes de S.L.O.T.H., ya que permite migrar lógica computacional desde hojas de cálculo hacia el motor de base de datos. Este componente interpreta fórmulas presentes en archivos .xlsx, construye representaciones intermedias mediante AST y grafos de dependencia, y genera sentencias SQL equivalentes cuando las funciones detectadas son soportadas.

Los resultados reportados muestran que, cuando el sistema de parsing migró el 30% de las fórmulas de una hoja de cálculo al motor de base de datos del data warehouse, los tiempos de carga se redujeron en un promedio de 9,7%. Más allá de esta reducción, el beneficio principal es arquitectónico: los cálculos dejan de depender exclusivamente del archivo cliente y pasan a ejecutarse en una infraestructura centralizada, trazable y reproducible.

Este resultado fortalece la gobernanza de datos, ya que las operaciones de negocio dejan de permanecer encapsuladas en celdas de hojas de cálculo y se convierten en instrucciones ejecutables por el motor de base

de datos. De esta manera, el sistema contribuye a reducir silos de conocimiento, mejorar la consistencia de los cálculos y facilitar auditoría sobre la lógica aplicada.

## 7.5 Resultados arquitectónicos

Desde el punto de vista arquitectónico, S.L.O.T.H. logra materializar una separación clara entre interacción de usuario, coordinación de negocio, procesamiento asincrónico, parsing, persistencia operacional y almacenamiento de reglas. Esta separación se refleja en cinco componentes principales: Web App, Core API, Typechecking Service, Parsing Subsystem y Database Service.

La arquitectura permite que las operaciones administrativas y de consulta se atiendan de forma sincrónica mediante HTTP/REST, mientras que las operaciones pesadas se delegan a RabbitMQ y workers especializados. A su vez, gRPC permite comunicación interna tipada y eficiente entre servicios core, especialmente en los flujos de parsing y acceso a estado operacional.

Este diseño cumple con los objetivos planteados para el proyecto: desacoplar reglas de negocio del código fuente, permitir procesamiento distribuido, mantener trazabilidad de tareas, separar responsabilidades entre servicios y facilitar la evolución independiente de cada componente. Además, la arquitectura permite que el sistema pueda escalar verticalmente mediante múltiples hilos y horizontalmente mediante múltiples instancias de workers o servicios.

## 7.7 Síntesis de resultados

En conjunto, los resultados muestran que S.L.O.T.H. cumple con los objetivos principales planteados para el proyecto. El sistema permite centralizar y versionar reglas de validación, reducir dependencia técnica mediante una interfaz no-code, validar archivos de forma asincrónica, procesar datos con mejoras significativas de rendimiento, migrar fórmulas de hojas de cálculo hacia SQL y desplegar la plataforma sobre una infraestructura reproducible y observable.

Los resultados empíricos evidencian mejoras de hasta 55% en tiempo total de ejecución, reducciones cercanas al 70% en la fase de validación y una disminución promedio del 9,7% en tiempos de carga cuando parte de las fórmulas es migrada al motor de base de datos. Además, la prueba de usabilidad de la interfaz muestra que usuarios de distintos perfiles pueden crear reglas de validación sin escribir código, aunque persiste una oportunidad de mejora en la construcción de patrones textuales complejos.

# 8 Conclusiones

S.L.O.T.H. responde a una problemática recurrente en los ecosistemas de Business Intelligence y Data Warehousing: la presencia de lógica de negocio dispersa en hojas de cálculo, scripts ETL y procesos dependientes de intervención técnica. Esta dispersión dificulta la trazabilidad, auditoría, modificación y reutilización de reglas, además de limitar la escalabilidad de los procesos de carga cuando los archivos aumentan en volumen o dimensionalidad.

El sistema desarrollado propone una solución integral basada en una arquitectura ETL distribuida, en la que las reglas de validación son centralizadas, versionadas y representadas mediante JSON Schema. Esta decisión permite desacoplar la lógica de validación del código fuente y administrarla como un artefacto independiente, auditable y modificable desde una interfaz gráfica. Como resultado, S.L.O.T.H. fortalece la

gobernanza de datos y reduce la dependencia de equipos técnicos para cambios operativos sobre reglas de negocio.

La arquitectura basada en microservicios permite separar responsabilidades entre los componentes de presentación, coordinación, validación, parsing, persistencia y observabilidad. Esta organización facilita la evolución del sistema, permite escalar servicios de forma independiente y reduce el acoplamiento entre componentes con necesidades técnicas distintas. En particular, la separación entre Core API, Typechecking Service, Parsing Subsystem y Database Service permite que las operaciones de usuario, procesamiento pesado y acceso a estado distribuido se manejen bajo mecanismos especializados.

Los resultados de rendimiento confirman la pertinencia del procesamiento paralelo y asincrónico. El uso de RabbitMQ y workers permite reducir significativamente los tiempos de validación y mejorar el rendimiento general de los flujos de carga. Las pruebas muestran que el sistema escala de forma predecible tanto frente al incremento de filas como frente al incremento de columnas, evitando comportamientos exponenciales en los escenarios evaluados.

El subsistema de parsing representa una contribución central del proyecto, ya que permite migrar lógica computacional contenida en hojas de cálculo hacia instrucciones SQL ejecutadas por el motor de base de datos. Esta capacidad no solo reduce parcialmente los tiempos de carga, sino que también mejora la trazabilidad y consistencia de los cálculos, al trasladarlos desde archivos manipulados por usuarios hacia una infraestructura centralizada y controlada.

Desde la perspectiva operativa, la infraestructura propuesta permite desplegar S.L.O.T.H. de forma reproducible mediante Terraform, Ansible, Docker Swarm y GitHub Actions. La incorporación de observabilidad, separación por ambientes, redes internas segmentadas, políticas de despliegue y automatización CI/CD demuestra que el proyecto no se limita a un prototipo funcional, sino que contempla elementos necesarios para una operación distribuida y mantenible.

En síntesis, S.L.O.T.H. logra articular una solución técnica coherente para la validación, carga y migración de lógica de negocio desde hojas de cálculo hacia un entorno ETL gobernado. Sus principales aportes se concentran en la centralización de reglas, el versionamiento de esquemas, la validación asincrónica, la traducción de fórmulas a SQL, la trazabilidad de tareas y la ejecución sobre una infraestructura escalable y observable.

## 8.1 Limitaciones

A pesar de los resultados obtenidos, el sistema mantiene limitaciones propias de su estado actual como MVP. El soporte de fórmulas de Excel traducibles a SQL no cubre la totalidad del lenguaje de fórmulas disponible en hojas de cálculo, por lo que algunas expresiones deben ser reportadas como no soportadas. Esta restricción evita traducciones incorrectas o parciales, pero limita el alcance de automatización en archivos con fórmulas complejas.

Asimismo, el flujo principal está orientado a PostgreSQL como motor destino inicial. Aunque la arquitectura permite pensar en extensiones hacia otros motores, la compatibilidad universal con bases de datos relacionales o analíticas no forma parte del alcance actual.

También existen oportunidades de mejora en la interfaz no-code. Los resultados de usabilidad muestran que las validaciones numéricas y booleanas son fácilmente comprensibles, pero las validaciones basadas en expresiones regulares siguen siendo complejas para usuarios no técnicos. Por tanto, la interfaz requiere mecanismos adicionales para construir patrones textuales de forma guiada.

## 8.2 Trabajo futuro

Como trabajo futuro, se propone ampliar progresivamente el soporte del subsistema de parsing para incluir más funciones, operadores y patrones de fórmulas de Excel. Esta ampliación permitiría migrar un porcentaje mayor de lógica computacional hacia SQL y reducir aún más la dependencia de archivos cliente.

También se plantea desarrollar un constructor visual de patrones o *Pattern Builder*, orientado a facilitar la creación de validaciones textuales sin exigir conocimiento de expresiones regulares. Esta mejora completaría de forma más robusta la visión no-code del sistema y respondería directamente a los resultados de usabilidad obtenidos.